

Thị giác chuỗi và không-thời gian (Sequential and Spatio-Temporal Vision)

PHỤ THUỘC C/13-kien-truc-backbone (CNN, attention); C/14-bai-toan-cv-loi (detection, segmentation — tracking dựng thẳng trên detection); B/11-gioi-han-tinh-toan (ngân sách bộ nhớ và tính toán); B/07-chia-du-lieu-va-danh-gia (chọn metric, rò rỉ dữ liệu). [T3]

TÓM TẮT

Chương này hỏi: thêm chiều thời gian vào ảnh thì phát sinh những vấn đề nào mà ảnh tĩnh không có? Câu trả lời là đúng ba vấn đề, và ba mục của chương tương ứng với ba vấn đề đó. Thứ nhất, đầu ra có độ dài thay đổi — đây là nơi *attention* ra đời. Thứ hai, chi phí bộ nhớ nhân theo số khung hình. Thứ ba, mô hình chạy trực tuyến không được nhìn khung hình tương lai. Đọc xong bạn thiết kế được một hệ thị giác thời gian thực bắt đầu từ ràng buộc chứ không từ bảng xếp hạng: biết kiến trúc nào bị loại ngay, ngân sách bộ nhớ phải chia thế nào, và metric nào không đánh lừa. Nếu chỉ nhớ một điều: hai bài toán khó nhất của thị giác thời gian là tương ứng điểm và liên kết dữ liệu. Đó đúng là hai bài toán mà một hệ SLAM giải ở mỗi khung hình. Phần lớn chương này vì thế là dịch từ vựng, không phải học lại từ đầu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **khung hình (frame) và clip** — khung hình là một ảnh tại một thời điểm; clip là một đoạn gồm T khung liên tiếp, và là đơn vị đầu vào của mô hình video — tương đương “một ảnh” của mô hình ảnh.
- **trực tuyến (online / streaming)** — chế độ chạy mà dữ liệu đến dần theo thời gian và phải trả kết quả ngay, khác với chế độ ngoại tuyến được xem toàn bộ video rồi mới quyết định.
- **nhân quả (causal)** — tính chất của một mô hình chỉ dùng thông tin tới thời điểm hiện tại. Mô hình hai chiều — nhìn cả trước lẫn sau — không nhân quả, nên không dùng được cho hệ trực tuyến.
- **độ trễ (latency) và thông lượng (throughput)** — độ trễ là thời gian từ lúc khung hình vào tới lúc có kết quả; thông lượng là số khung xử lý được mỗi giây. Hai đại lượng này độc lập: gộp batch làm thông lượng tăng nhưng độ trễ tăng theo.
- **optical flow** — trường vector cho biết mỗi pixel dịch chuyển đi đâu giữa hai khung hình — biểu diễn tường minh của chuyển động, và là dạng dày đặc của bài toán tương ứng điểm.
- **tracking và track ID** — tracking là giữ danh tính của từng đối tượng qua thời gian; track ID là mã số phải giữ nguyên suốt thời gian đối tượng xuất hiện. Đánh tráo mã số giữa hai đối tượng gọi là *ID switch*.
- **attention** — cơ chế cho bên cần thông tin tính một bộ trọng số trên các phần của bên có thông tin rồi lấy tổ hợp có trọng số — tức hoãn việc nén dữ liệu lại cho tới khi biết đang cần gì.

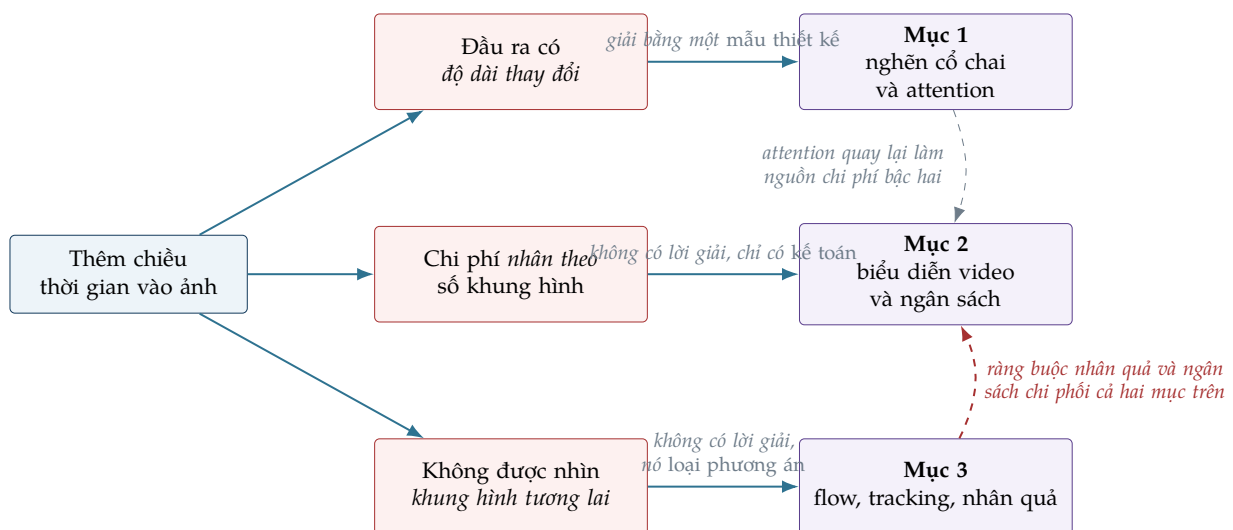
CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt mIoU cao trên từng khung hình vẫn cho ra video nhấp nháy, và vì sao không có thước đo phổ biến nào bắt được lỗi đó?

- Q2.** *attention* thường được kể như một phát minh của Transformer. Vậy nó thực sự ra đời để giải quyết nút thắt nào, và vì sao nút thắt đó xuất hiện lần đầu ở bài toán mô tả ảnh chứ không ở phân loại?
- Q3.** Một hệ tracking có thể đánh tráo danh tính đối tượng liên tục mà chỉ số MOTA gần như không suy chuyển. Công thức nào cho phép chuyển đó xảy ra?
- Q4.** Vì sao “thời gian là trục thứ ba của tensor” là một câu nói sai, dù về mặt lập trình nó hoàn toàn đúng?
- Q5.** Nếu bạn tăng gấp bốn độ dài clip mà giữ nguyên bộ nhớ, bạn phải trả bằng gì — và vì sao ba cách trả giá đó không hề tương đương?

Chương 14 xử lý từng ảnh như một ốc đảo. Đó là một đơn giản hoá rất tiện nhưng rất xa thực tế. Camera không sinh ra ảnh rời rạc mà sinh ra một dòng liên tục. Phần lớn thông tin hữu ích chỉ tồn tại *giữa* các khung hình, không nằm trong khung nào cả. Một vật đang tiến lại gần, một người vừa ngã, cùng một chiếc xe hay hai chiếc khác nhau — không câu nào trong số đó trả lời được từ một ảnh tĩnh.

Thêm chiều thời gian không phải là thêm một chiều nữa cho tensor. Nó sinh ra đúng ba vấn đề mới, và ba vấn đề đó độc lập với nhau tới mức mỗi cái đáng một mục riêng. Thứ nhất, **đầu ra có thể có độ dài thay đổi**: mô tả một cảnh cần một câu, không phải một nhãn, và việc phải nén cả ảnh vào một vector trước khi sinh câu là nút thắt đã khai sinh ra *attention* trong thị giác. Thứ hai, **chi phí nhân theo số khung hình**. Ngân sách bộ nhớ của một mô hình ảnh vốn đã chật, giờ bị chia tiếp cho ba chiều: độ dài clip, độ phân giải, batch size. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Thứ ba, **ràng buộc nhân quả**: hệ chạy trực tuyến không được nhìn khung hình tương lai, và ràng buộc đó loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng.



Hình 1.1. Bản đồ chương: ba mục không phải ba chủ đề rời mà là ba *loại vấn đề* khác nhau mà chiều thời gian sinh ra, và ba loại câu trả lời khác nhau về bản chất. Mũi tên đứt cho thấy ba nhánh không độc lập: ràng buộc nhân quả và ngân sách bộ nhớ quyết định cách chữa nào ở mục 3 là khả thi, còn *attention* của mục 1 quay lại chính là nguồn chi phí bậc hai của mục 2.

Hình 1.1 nên xem trước khi đọc, vì nó cho biết mục nào trả lời loại câu hỏi nào — và vì thế mục nào đáng đọc kỹ với bài toán cụ thể của bạn.

Ba mục của chương lần lượt xử lý ba vấn đề đó, theo thứ tự phụ thuộc chứ không theo thứ tự lịch sử. Mục 1 đi từ sinh mô tả ảnh kiểu CNN–RNN tới “Show, Attend and Tell”. Nên đọc nó *trước* phần Transformer của chương 13. Lý do là nó cho thấy *attention* ra đời để gỡ một nút thắt cụ thể, không phải để làm kiến trúc đẹp hơn. Mục 2 chuyển sang biểu diễn video và biến ngân

sách bộ nhớ thành những con số tính được bằng tay, vì đó là ràng buộc mà mọi quyết định phía sau phải sống chung. Mục 3 gom ba bài toán mà một kỹ sư robot gặp hằng ngày — optical flow, tracking, và tính nhất quán thời gian — rồi kết thúc bằng ràng buộc nhân quả, thứ chi phối tất cả.

Với người đọc đến từ hình học và SLAM, chương này gần chuyên môn hơn bất kỳ chương nào khác trong Phần C, và cách đọc hiệu quả nhất là đọc như một cuốn từ điển đối chiếu. Optical flow là bài toán tương ứng điểm ở dạng dày đặc, với đúng bài toán khâu độ và đúng ma trận cấu trúc mà bạn dùng để chọn đặc trưng góc. Tracking là bài toán liên kết dữ liệu, với đúng bộ đôi dự đoán bằng bộ lọc trạng thái cộng gán tối ưu. Còn ràng buộc nhân quả và ngân sách mili-giây thì bạn đã sống cùng nó hằng ngày. Cái mới ở đây không phải cấu trúc bài toán mà là các thành phần học được đặt vào từng vị trí, cùng với câu hỏi quen thuộc: mỗi lần thay một khối hình học bằng một khối học được thì ta mua được độ bền gì và mất đi tính chất gì.

Bảng 1.1. Từ điển đối chiếu cho người đến từ SLAM. Cột giữa và cột phải là cùng một bài toán với hai truyền thống thuật ngữ khác nhau; chỉ cột cuối là thứ thực sự mới.

Bài toán	Tên trong SLAM / hình học	Tên trong thị giác học sâu, và cái mới
Tương ứng điểm	Ghép đặc trưng, KLT, tìm kiếm dọc đường epipolar	Optical flow dày đặc; cái mới là học được prior về chuyển động nên vùng không kết cấu cũng có đáp án
Liên kết dữ liệu	Ghép đặc trưng với điểm bản đồ, công lọc Mahalanobis, RANSAC	Tracking-by-detection; cái mới là dùng thêm đặc trưng ngoại hình học được làm chi phí ghép
Dự đoán trạng thái	Bộ lọc Kalman, mô hình chuyển động hằng vận tốc	Vẫn là Kalman trong đa số bộ tracker; cái mới nằm ở phần đo, không ở phần dự đoán
Chọn đặc trưng tốt	Giá trị riêng nhỏ nhất của ma trận cấu trúc	Cùng ma trận đó xuất hiện trong Lucas–Kanade; mô hình học sâu thay tiêu chí này bằng khối tương quan
Nhận diện nơi đã qua	Túi từ trực quan, kiểm chứng hình học	Nhúng toàn cục cộng bước quay lại đặc trưng — đúng cấu trúc của attention
Ngân sách thời gian thực	Giới hạn số vòng lặp tối ưu, kích thước cửa sổ trượt	Số bước tinh chỉnh của RAFT, độ dài clip, cửa sổ nhân quả

Bảng 1.1 nên đọc trước cả ba mục. Nó cho biết chỗ nào bạn được phép đọc lướt vì cấu trúc bài toán đã quen. Nó cũng chỉ ra chỗ phải đọc kỹ: nơi một thành phần học được thay vào, mang theo giả định mới.

Bản đồ chương 15 — Thị giác chuỗi và không-thời gian

TÓM TẮT

Chương 15 chia làm ba mục theo đúng ba vấn đề mà chiều thời gian sinh ra, chứ không theo thứ tự lịch sử của các kiến trúc. Ba vấn đề đó là: đầu ra có độ dài thay đổi, chi phí bộ nhớ nhân theo số khung hình, và ràng buộc nhân quả. Bản đồ này nói mục nào cốt lõi, mục nào tra khi cần, và mục nào nên đọc chen vào giữa chương 13. Nếu chỉ nhớ một điều: mục 3 là mục gần chuyên môn của một kỹ sư SLAM nhất và cũng là mục dùng được ngay nhất, còn mục 1 nên đọc *trước* phần Transformer của chương 13.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chương và mục** — chương là một chủ đề lớn, mỗi mục là một file đọc độc lập được. Số trong tên thư mục là thứ tự nên đọc, không phải thứ tự lịch sử.
- **trực tuyến (streaming) và ngoại tuyến** — trực tuyến là xử lý dữ liệu đến dần và trả kết quả ngay; ngoại tuyến là được xem toàn bộ video rồi mới quyết định. Phần lớn số liệu công bố là ngoại tuyến.
- **nhân quả (causal)** — chỉ dùng thông tin tới thời điểm hiện tại, không nhìn khung hình tương lai — ràng buộc cứng của mọi hệ chạy trên robot.
- **ngân sách bộ nhớ ba chiều** — lượng bộ nhớ khi huấn luyện video tỉ lệ với độ dài clip $\times$ độ phân giải² $\times$ batch size, nên tăng một chiều buộc phải giảm chiều khác.

Có gì trong chương này

- **Mục 1 — Đầu ra độ dài thay đổi và sự ra đời của attention** (*cốt lõi, đọc sớm*). Sinh mô tả ảnh kiểu CNN-RNN, nghẽn cổ chai của một vector duy nhất, và “Show, Attend and Tell”. Giá trị lớn nhất của mục không phải bài toán captioning mà là việc nó cho thấy *động cơ* của attention.
- **Mục 2 — Biểu diễn video và cái giá của thời gian** (*cốt lõi*). Gộp theo khung, hai luồng, tích chập 3D, bản tách (2+1)D, SlowFast, video transformer — và ba phép tính bộ nhớ quyết định tất cả.
- **Mục 3 — Flow, tracking và các ràng buộc riêng** (*cốt lõi, gần chuyên môn nhất*). Lucas-Kanade tới RAFT, định vị hành động theo khoảng thời gian, tracking như bài toán liên kết dữ liệu, bẫy của MOTA và cách HOTA sửa nó, tính nhất quán thời gian, và ràng buộc nhân quả.

Vì sao chia như vậy

Cách chia hiển nhiên sẽ là chia theo bài toán — captioning, phân loại hành động, tracking — nhưng cách đó che mất điều đáng học. Ba mục ở đây chia theo *ba vấn đề cấu trúc* mà chiều thời gian sinh ra, và mỗi vấn đề có một dạng câu trả lời khác hẳn. Vấn đề thứ nhất được giải bằng một mẫu thiết kế, tức attention. Vấn đề thứ hai không có lời giải, chỉ có kế toán. Vấn đề thứ ba cũng không có lời giải; nó chỉ loại bỏ phương án. Nhìn theo trục này thì các kiến trúc cụ thể trở thành hệ quả, và bạn không phải nhớ chúng như một danh sách.

Thứ tự đọc và phụ thuộc

1. Đọc mục 1 *trước* C/13-kien-truc-backbone/02-tu-chuoi-toi-attention: hiểu nút thắt trước thì self-attention thành hệ quả hiển nhiên thay vì một phát minh khó hiểu.
2. Mục 2 trước mục 3: ngân sách bộ nhớ ở mục 2 là thứ quyết định cách chữa nào cho hiện tượng nhấp nháy là khả thi ở mục 3.
3. Mục 3 giả định bạn đã đọc C/14-bai-toan-cv-loi, vì tracking-by-detection dựng thẳng trên một detector.
4. Đọc B/07-chia-du-lieu-va-danh-gia song song với mục 3 nếu phần metric và rò rỉ dữ liệu theo clip còn mờ.
5. C/16-thi-giac-3d là chương tiếp nối tự nhiên: nó nâng bài toán tương ứng điểm từ mặt phẳng ảnh lên không gian ba chiều.

Nếu chỉ có một buổi

Đọc mục 3, và trong mục 3 đọc kỹ hai chỗ. Thứ nhất là bài toán khâu độ với ma trận cấu trúc: nó nổi thẳng vào việc chọn đặc trưng mà bạn đã làm. Thứ hai là hộp cạm bẫy về MOTA: nó thay đổi ngay cách bạn báo cáo kết quả tracking. Phần còn lại có thể đọc lướt và quay lại khi cần.

1.1 Đầu ra độ dài thay đổi và sự ra đời của attention (Variable-Length Outputs and the Birth of Attention)

PHỤ THUỘC C/13-kien-truc-backbone/02-tu-chuoi-toi-attention (RNN, LSTM, attention — mục này là *động cơ* của mục đó, nên đọc mục này trước sẽ dễ hơn); B/05-thiet-ke-bai-toan (dạng đầu ra).

TÓM TẮT

Mục này kể lại đúng một sự kiện: **attention** không sinh ra từ mong muốn làm kiến trúc đẹp hơn, mà sinh ra để gỡ một nút thắt rất cụ thể — việc nén cả một ảnh vào một vector duy nhất trước khi sinh câu mô tả. Đọc xong bạn hiểu *vì sao* self-attention của Transformer là hiển nhiên chứ không bí ẩn, và bạn nhận ra được cùng một nút thắt ở những nơi khác, kể cả trong pipeline SLAM của chính mình. Nếu chỉ nhớ một điều: ở đâu một hệ thống nén thông tin có cấu trúc thành một vector cố định *trước khi* biết sẽ cần phần nào, ở đó có một nút thắt. Attention là cách chuẩn để gỡ nó, bằng cách hoãn phép nén tới lúc biết đang cần gì.

KIẾN THỨC CẦN CÓ TRƯỚC

- **token** — một đơn vị rời rạc của chuỗi đầu ra — thường là một từ hoặc một mảnh từ. Mô hình sinh câu bằng cách dự đoán token tiếp theo, lặp lại tới khi gặp token kết thúc.
- **RNN và LSTM** — mạng xử lý chuỗi bằng cách mang theo một *trạng thái ẩn* (một vector) từ bước này sang bước sau. LSTM là biến thể có cổng, giúp trạng thái giữ được thông tin qua nhiều bước hơn.
- **encoder-decoder** — encoder biến đầu vào thành biểu diễn số; decoder sinh đầu ra từ biểu diễn đó. Ở đây encoder là CNN nhìn ảnh, decoder là mạng sinh câu.
- **feature map (lưới đặc trưng)** — đầu ra không gian của CNN: một lưới, ví dụ 14×14 , mỗi ô là một vector mô tả một vùng ảnh. Giữ nguyên lưới nghĩa là giữ nguyên thông tin “ở đâu”.
- **global average pooling** — phép lấy trung bình mọi ô của lưới đặc trưng để ra một vector duy nhất. Rất tiện cho phân loại, nhưng nó xóa sạch thông tin vị trí.
- **attention** — cơ chế cho một bên (bên cần thông tin) tính một bộ trọng số trên các phần của bên kia (bên có thông tin), rồi lấy tổ hợp có trọng số. Trọng số phụ thuộc vào việc đang cần gì, nên phép nén được “hoãn” tới lúc có truy vấn.
- **self-attention** — trường hợp hai bên là cùng một tập: mỗi phần tử tự tính trọng số trên mọi phần tử khác của chính chuỗi đó. Đây là khối cơ bản của Transformer.

Đặt vấn đề

Toàn bộ chương 14 giả định một điều mà ta không để ý: đầu ra tuy có thể là tập hợp nhưng *mỗi phần tử* của nó vẫn có hình dạng cố định — một khung là bốn số, một mask là một lưới nhị phân. **Sinh mô tả ảnh** (*image captioning*) phá vỡ cả điều đó: đầu ra là một câu, dài ngắn tùy ảnh, và mỗi từ phụ thuộc vào các từ trước. Đây là lần đầu thị giác máy tính phải sinh ra một chuỗi, và nó buộc phải mượn bộ máy của xử lý ngôn ngữ.

Giải pháp đầu tiên nghe rất hợp lý. Cho ảnh qua CNN, lấy vector đặc trưng ở tầng áp chót, dùng nó làm trạng thái khởi tạo cho một **mạng hồi quy** (*recurrent neural network, RNN*) hoặc **LSTM** [1], rồi để mạng đó sinh từng **token** một. Kiến trúc này chạy được và cho ra những câu nghe khá trôi chảy. Nhưng nó chứa một giả định chết người: rằng *một* vector cố định — thường 512 hay 2048 số — đủ để trả lời mọi câu hỏi mà mọi từ trong câu sẽ đặt ra. Khi mạng đang sinh từ “con chó”, nó cần biết chi tiết ở góc dưới bên trái; khi sinh từ “bãi cỏ”, nó cần chi tiết ở nền. Vector kia phải chứa sẵn cả hai, và cả những chi tiết cho mọi câu khác mà nó không biết trước.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn phát hiện được nút thắt kiểu “nén trước, dùng sau” trong một kiến trúc bất kỳ, kể cả kiến trúc không dính gì tới ngôn ngữ. Bạn cũng biết attention là công cụ chuẩn để gỡ nó, và biết cái giá phải trả khi gỡ.

TRỰC GIÁC

Hãy tưởng tượng bạn phải kể lại một bộ phim cho người khác, nhưng luật chơi là: xem xong, bạn chỉ được ghi *một* câu tóm tắt vào giấy, rồi trả lại bằng phim, và từ đó chỉ được nhìn tờ giấy để kể suốt hai mươi phút. Bạn sẽ kể trôi chảy, nhưng mọi chi tiết đều là bịa hợp lý. Attention là việc bỏ luật đó đi: mỗi khi kể đến một cảnh, bạn được phép tua lại băng và nhìn đúng cảnh ấy. Không có gì huyền bí — toàn bộ ý tưởng chỉ là *hoãn việc nén lại cho tới khi biết mình đang cần gì*.

A. Từ một vector tới một cái nhìn có chọn lọc

I Mạch 1 — đầu ra độ dài thay đổi

- 1 Vấn đề** đầu ra là một câu, không phải một nhãn.

Giải pháp gán một bộ giải mã chuỗi lên trên bộ trích đặc trưng: CNN cho vector, RNN/LSTM sinh từng token, huấn luyện bằng **entropy chéo** (*cross-entropy*) trên từng vị trí token.

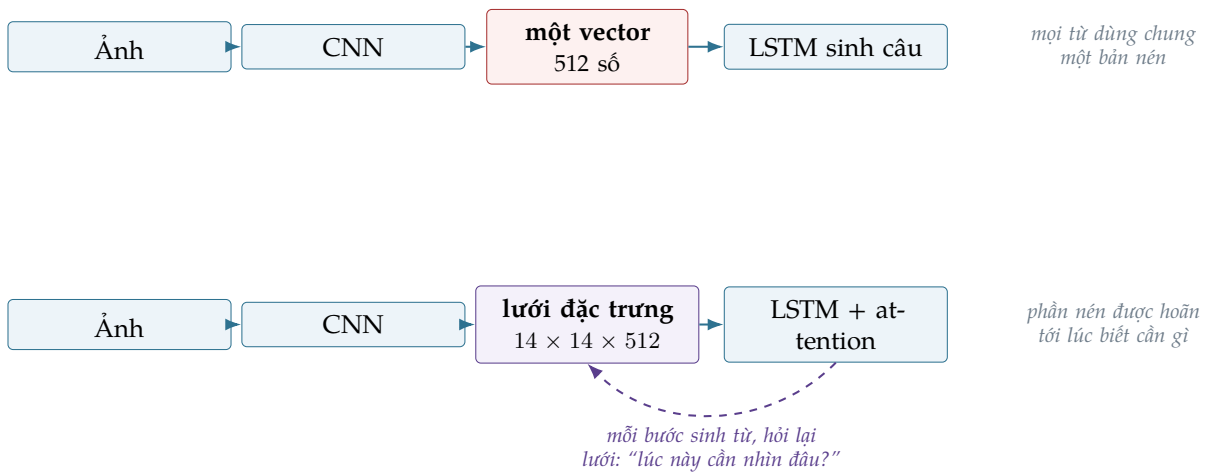
Giả định đặc trưng toàn cục của ảnh chứa đủ thông tin cho toàn bộ câu.

Hệ quả lần đầu tiên một hệ thị giác sinh được đầu ra có độ dài thay đổi.
- 2 Vấn đề** một vector duy nhất là nghẽn cổ chai thông tin. Mọi chi tiết không gian bị nén trước khi ta biết sẽ cần chi tiết nào; và nén *có mất mát* thì không đảo ngược được.

Giải pháp “Show, Attend and Tell” giữ nguyên lưới đặc trưng không gian [2]. Ở mỗi bước sinh từ, bộ giải mã tính một phân bố trọng số trên các ô lưới. Tổ hợp có trọng số của các ô đó là đầu vào cho bước ấy.

Hệ quả đây là nơi attention xuất hiện lần đầu trong thị giác, và nó ra đời để gỡ một nút thắt cụ thể chứ không phải vì đẹp. Ý tưởng gốc đến từ dịch máy, nơi cùng một nghẽn cổ chai xuất hiện ở dạng vector câu nguồn [3].
- 3 Vấn đề** nếu attention hữu ích cho quan hệ giữa *câu và ảnh*, tại sao không dùng cho quan hệ giữa *các phần của cùng một ảnh*?

Giải pháp đó chính là bước đi tới self-attention và **transformer** [4], và đó là lý do nên đọc mục này *trước* C/13-kien-truc-backbone/02-tu-chuoi-toi-attention: khi đã thấy nút thắt, self-attention trở thành hệ quả hiển nhiên thay vì một phát minh từ trên trời rơi xuống.



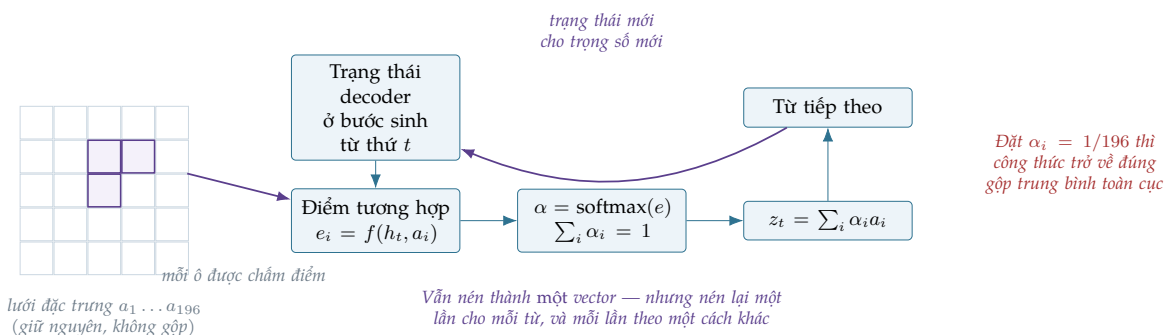
Hình 1.2. Cùng một pipeline, khác nhau đúng một chỗ. Nhánh trên nén lưới đặc trưng thành một vector trước khi biết câu sẽ nói gì; nhánh dưới giữ nguyên lưới và để bộ giải mã truy vấn lại nó ở mỗi bước. Mũi tên đứt là toàn bộ nội dung của attention — một vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

B. Nút thắt lớn tới mức nào — một phép tính

Hình 1.2 sẽ thuyết phục hơn nhiều khi kèm con số. Lưới đặc trưng của một CNN trước tầng gộp toàn cục có kích thước điển hình $14 \times 14 \times 512$, tức $196 \times 512 = 100\,352$ số. Sau **gộp trung bình toàn cục** (*global average pooling*) chỉ còn 512 số. Tỉ số nén là $196:1$, và điều quan trọng không phải tỉ số mà là *cái gì bị mất*: phép gộp lấy trung bình theo không gian, nên nó xoá sạch thông tin “đặc trưng này nằm ở đâu”. Một câu mô tả cần đúng thông tin đó để nói “con chó bên trái người đàn ông”.

↔ **hệ quả của C/13/02** — self-attention là bước tiếp theo hiển nhiên khi đem chính cơ chế này áp cho quan hệ giữa các phần bên trong một đầu vào, thay vì giữa hai đầu vào khác nhau.

Nói gọn thì attention thay phép trung bình đều bằng phép trung bình có trọng số, với trọng số phụ thuộc vào việc bộ giải mã đang cần gì: gọi a_i là vector đặc trưng của ô thứ i , và α_i là trọng số attention tại bước sinh từ hiện tại. Đầu vào của bước đó là $z = \sum_i \alpha_i a_i$, với $\sum_i \alpha_i = 1$. Công thức này nói một điều rất đơn giản: *vẫn nén thành một vector, nhưng nén lại một lần cho mỗi từ, và mỗi lần nén theo một cách khác nhau*. Trường hợp $\alpha_i = 1/196$ với mọi i chính là gộp trung bình toàn cục — mô hình cũ là một trường hợp riêng của mô hình mới.



Hình 1.3. Cơ chế attention viết ra thành luồng tính. Điều đáng chú ý không phải công thức mà là vòng phản hồi: trạng thái của bộ giải mã quyết định trọng số, trọng số quyết định vector đưa vào bước sau, và bước sau lại đổi trạng thái. Mô hình cũ là trường hợp riêng khi trọng số bị ép bằng nhau, nên đây là một phép tổng quát hoá chứ không phải một kiến trúc cạnh tranh.

Hình 1.3 cũng cho thấy chi phí: phần chấm điểm phải chạy lại cho *mỗi* từ trên *mọi* ô lưới, nên tổng chi phí tỉ lệ với tích số ô nhân số bước sinh — mằm mông của vấn đề bậc hai ở mục sau.

Bảng 1.2. Ba cách đưa thông tin ảnh vào bộ giải mã chuỗi.

Cách	Cơ chế	Giá phải trả	Hỏng khi nào
Một vector toàn cục	Gộp trung bình rồi khởi tạo trạng thái	Rẻ nhất, một lần tính	Câu cần chi tiết cục bộ hoặc quan hệ không gian; mô hình bịa chi tiết hợp lý
Soft attention	Trung bình có trọng số, khả vi	Chi phí tỉ lệ số ô $\times$ số từ	Ảnh nhiều vật giống nhau: trọng số dàn đều, không chọn được
Hard attention	Lấy mẫu rồi rạc một ô	Rẻ khi chạy, nhưng không khả vi	Phải huấn luyện bằng ước lượng gradient có phương sai cao; kém ổn định

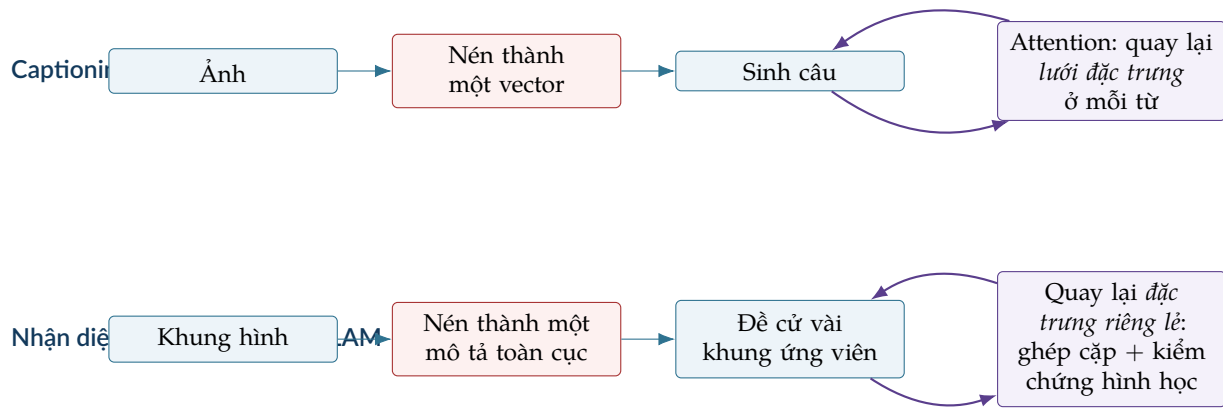
↔ cùng nguyên lý với C/14/02 — mỗi truy vấn của DETR “hỏi” feature map xem vật của nó ở đâu — đúng vòng phản hồi từ nơi cần thông tin về nơi có thông tin.

C. Cùng một nút thắt trong pipeline SLAM

Chỗ nổi này đáng dừng lại, vì nó cho thấy nút thắt trên không phải chuyện riêng của ngôn ngữ. Một hệ **SLAM** nhận diện nơi đã đi qua bằng cách nén cả khung hình thành một mô tả toàn cục — túi từ trực quan hoặc một vector nhúng — rồi so mô tả đó với cơ sở dữ liệu [5], [6]. Bước nén ấy có *đúng* tính chất của nghẽn cổ chai: nó xảy ra trước khi biết sẽ cần kiểm chứng điều gì, nên nó vứt bỏ hình học.

Và giải pháp mà cộng đồng SLAM dùng cũng có đúng cấu trúc của attention: sau khi mô tả toàn cục đề cử vài ứng viên, hệ thống *quay lại* các đặc trưng riêng lẻ để ghép cặp và kiểm chứng bằng ràng buộc hình học. Nói cách khác, mô tả toàn cục chỉ dùng để *chọn nơi cần nhìn kỹ*, chứ không dùng để kết luận — y hệt vai trò của phân bố α trong công thức trên. Nếu bạn thấy self-attention khó hình dung, hãy nghĩ nó như bước kiểm chứng đặc trưng đó, chỉ khác là trọng số được học thay vì được tính bằng RANSAC.

↔ cùng nguyên lý với C/16 — nhận diện nơi đã đi qua nén khung hình thành một mô tả toàn cục rồi phải quay lại đặc trưng để kiểm chứng hình học — cùng cấu trúc “đề cử rồi nhìn kỹ” của attention.



Cùng một cấu trúc: bản nén toàn cục chỉ dùng để chọn nơi cần nhìn kỹ, không dùng để kết luận. Khác biệt duy nhất là trọng số quay lại được học (attention) hay được tính bằng ràng buộc hình học (RANSAC).

Hình 1.4. Nghẽn cổ chai “nén trước, dùng sau” không phải chuyện riêng của ngôn ngữ. Hệ SLAM gặp đúng nó khi nén một khung hình thành một mô tả toàn cục để tra cứu, và giải đúng theo cách của attention: dùng bản nén để đề cử, rồi quay về nguồn thông tin đầy đủ để kiểm chứng. Nhận ra được cấu trúc này thì self-attention thôi trông giống một phát minh xa lạ.

Hình 1.4 là lý do mục này nên đọc trước phần Transformer: khi mẫu thiết kế đã quen, phần công thức chỉ còn là chi tiết.

D. Giới hạn và trường hợp hỏng

- **Thước đo của captioning rất yếu.** Các độ đo dựa trên trùng khớp n -gram thường cho câu “an toàn”: đúng ngữ pháp và giống câu tham chiếu. Vì thế một mô hình sinh mô tả chung chung có thể ghi điểm cao hơn một mô hình mô tả đúng nhưng diễn đạt khác. *Hệ quả thực tế:* cải tiến trên bảng xếp hạng không bảo đảm cải tiến khi người dùng thật đọc.
- **Mô hình bịa chi tiết.** Vì bộ giải mã được huấn luyện để sinh câu *hợp lý*, nó lấp chỗ trống bằng thứ thường đúng — “người đàn ông cầm ván lướt sóng” cho một ảnh bãi biển không có ván. Nút thắt càng chặt thì hiện tượng này càng nặng, nên đây là triệu chứng chẩn đoán được.
- **Bản đồ attention không phải lời giải thích.** Rất dễ nhìn α rồi tin rằng mô hình “đang nhìn vào đó vì lý do đó”. Các kiểm tra tính đúng đắn cho thấy nhiều loại bản đồ nổi bật vẫn trông hợp lý ngay cả khi trọng số mô hình bị làm hỏng [7], nên trọng số attention là *manh mối chẩn đoán*, không phải bằng chứng.
- **Chi phí tăng theo tích số ô $\times$ số bước.** Với ảnh phân giải cao hoặc câu dài, phần attention có thể vượt chi phí của chính CNN. Đây là mầm mống của vấn đề bậc hai ở mục sau, khi số ô còn nhân thêm với số khung hình.

CẦN NHỚ

Nghẽn cổ chai thông tin xuất hiện ở *mọi* chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định trước khi biết sẽ cần phần nào. Attention không phải một loại tầng mạng, nó là một mẫu thiết kế: *giữ nguyên nguồn, và hoãn việc nén tới lúc có truy vấn*. Nhận ra mẫu này rồi thì Transformer, cross-attention trong DETR, và cả bước kiểm chứng hình học trong SLAM đều là cùng một thứ. [T2]

E. Kết nối

C/13-kien-truc-backbone/02-tu-chuoi-toi-attention là nơi hình thức hoá đầy đủ; mục này là động cơ của nó, nên thứ tự đọc tự nhiên là mục này trước.

C/14-bai-toan-cv-loi/02-detection-mot-giai-doan dùng lại chính cơ chế này dưới dạng truy vấn của DETR: mỗi truy vấn “hỏi” feature map xem vật của nó ở đâu. [C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là bước kế tiếp trực tiếp: khi nguồn không chỉ là một lưới không gian mà là một khối không-thời gian, chi phí attention nhân lên và trở thành ràng buộc thiết kế chính.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao nén ảnh thành một vector *trước* khi sinh câu lại là một quyết định tồi, trong khi nén ảnh thành một vector trước khi phân loại thì hoàn toàn ổn?
2. Chứng minh bằng một câu rằng gộp trung bình toàn cục là trường hợp riêng của soft attention. Điều đó nói gì về quan hệ giữa hai kiến trúc?
3. Hard attention rẻ hơn khi chạy nhưng ít được dùng. Vì sao?
4. Vì sao không nên đọc bản đồ attention như một lời giải thích về nguyên nhân?
5. Trong một hệ thống bạn đã làm, nghẽn cổ chai kiểu “nén trước, dùng sau” nằm ở đâu, và hệ thống đó bù lại bằng bước “quay lại nguồn” nào?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [5] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [7] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [8] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [9] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [10] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [11] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [12] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [13] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [14] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [15] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [16] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [17] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [18] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [19] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.

1.2 Biểu diễn video và cái giá của thời gian (Video Representation and the Cost of Time)

PHỤ THUỘC C/13/01-dong-cnn (conv, số tham số, FLOPs); C/13/02-tu-chuoi-toi-attention (chi phí bậc hai của attention); B/11-gioi-han-tinh-toan (bảng thông bộ nhớ, ngân sách tính toán).

TÓM TẮT

Mục này trả lời câu hỏi “thêm chiều thời gian vào ảnh thì phải trả giá gì”. Câu trả lời ngắn: bộ nhớ, và nó bị chia cho ba chiều cùng lúc — độ dài clip, độ phân giải, và **batch** — nên tăng một chiều buộc phải giảm chiều khác. Bốn họ kiến trúc (gộp theo khung, hai luồng, tích chập 3D và bản tách $(2+1)D$, video transformer) là bốn cách khác nhau để mua thông tin thời gian với ít bộ nhớ nhất có thể. Đọc xong bạn tính được ngân sách bộ nhớ của một cấu hình video *trước khi* chạy, và biết chiều nào nên hy sinh cho bài toán của mình. Nếu chỉ nhớ một điều: trước khi tin bất kỳ mô hình video nào, hãy chạy baseline “mỗi khung một lần rồi gộp”. Trên rất nhiều tập dữ liệu, baseline đó gần bằng mô hình chuyển động. Chênh lệch còn lại mới là thứ đáng trả tiền.

KIẾN THỨC CẦN CÓ TRƯỚC

- **clip** — một đoạn video ngắn gồm T khung hình liên tiếp (hoặc lấy mẫu thưa), là đơn vị mà mô hình video nhận vào — tương đương “một ảnh” của mô hình ảnh.
- **kích hoạt (activation)** — tensor giá trị trung gian mà mỗi tầng mạng sinh ra — khác với *hàm kích hoạt*, là phép phi tuyến áp lên nó. Khi huấn luyện phải giữ các tensor này trong bộ nhớ để tính gradient ở **lượt ngược** (*backward pass*), nên chúng — chứ không phải tham số — thường là thứ làm tràn bộ nhớ.
- **batch size** — số mẫu xử lý cùng lúc trong một bước cập nhật. Batch lớn cho ước lượng gradient ổn định hơn nhưng tốn bộ nhớ tỉ lệ thuận.
- **batch normalization** — phép chuẩn hoá kích hoạt bằng trung bình và phương sai *tính trong batch*. Batch quá nhỏ thì hai thống kê này rất nhiễu, nên mô hình video hay phải đổi sang chuẩn hoá theo nhóm hoặc theo tầng.
- **FLOPs** — số phép tính dấu phẩy động cần cho một lần chạy; dùng để so chi phí tính toán độc lập với phần cứng. Nó không tỉ lệ chặt với thời gian chạy thật, vì bộ nhớ mới hay là nút thắt.
- **optical flow** — trường vector cho biết mỗi pixel dịch đi đâu giữa hai khung hình liên tiếp — biểu diễn tường minh của chuyển động, dùng làm đầu vào cho nhánh chuyển động của kiến trúc hai luồng.
- **token và chi phí bậc hai** — Transformer chia đầu vào thành n token rồi tính quan hệ giữa mọi cặp, nên chi phí tỉ lệ n^2 . Với video, n nhân thêm số khung hình, và n^2 nhân thêm bình phương số đó.
- **inductive bias** — giả định cài sẵn trong kiến trúc, ví dụ tích chập giả định tính cục bộ và **đẳng biến tịnh tiến** (*translation equivariance*). Bỏ bớt giả định thì mô hình linh hoạt hơn nhưng cần nhiều dữ liệu hơn để tự học lại điều đã bị bỏ.

Đặt vấn đề

Cách hiển nhiên nhất để xử lý video là coi nó như một khối ba chiều và dùng **tích chập ba chiều** (*3D convolution*) thay cho tích chập hai chiều. Cách này đúng về mặt hình thức và sai về mặt kinh tế: số phần tử phải giữ trong bộ nhớ nhân với số khung hình, số tham số của mỗi kernel nhân với chiều dài thời gian, và ngân sách vốn đã chật của một mô hình ảnh lập tức vỡ. Vì thế toàn bộ lĩnh vực này không phải là câu chuyện “kiến trúc nào tốt hơn” mà là câu chuyện “mua thông tin thời gian bằng cách nào cho rẻ”.

Nhưng còn một lý do sâu hơn khiến ta không nên coi thời gian là trục không gian thứ ba. Trục thời gian có *hướng*: đảo ngược nó thì “mở cửa” thành “đóng cửa”, trong khi lật ngược trục x của một ảnh vẫn cho một ảnh hợp lệ của cùng một cảnh. Trục thời gian cũng có *đơn vị khác*: 16 khung hình ở 30 Hz là nửa giây, còn 16 pixel là một chi tiết nhỏ — không có lý do gì để cùng một kernel

$3 \times 3 \times 3$ là kích thước hợp lý ở cả ba trục. Hai bất đối xứng này là nguyên nhân sâu xa của mọi thiết kế tách trục mà ta sẽ gặp.

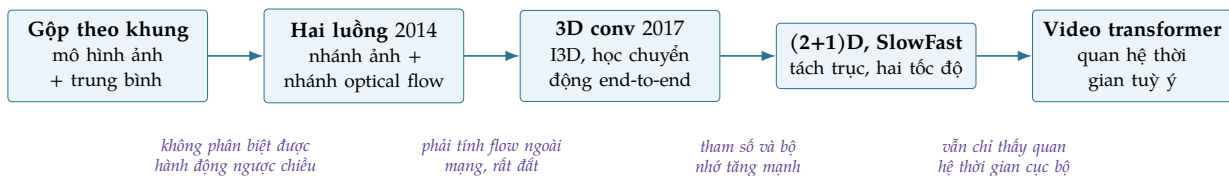
Học xong mục này bạn làm được gì mà trước đó không làm được? Cầm một yêu cầu như “phân loại hành động trên clip 4 giây, chạy được trên GPU 8 GB”, bạn suy ra ngay cấu hình khả thi và thứ phải hy sinh. Không phải thử ngẫu nhiên tới khi hết bộ nhớ.

TRỰC GIÁC

Ngân sách bộ nhớ của video giống một tấm chăn ngắn kéo theo ba hướng: độ dài clip, độ phân giải, và batch size. Tích của ba đại lượng đó gần như là một hằng số do phần cứng ấn định, nên mọi quyết định kiến trúc thực chất là quyết định kéo chăn về phía nào. Điều đáng nói là ba hướng ấy không tương đương về mặt khoa học: giảm batch làm hỏng thống kê của **chuẩn hoá theo batch** (*batch normalization*), giảm độ phân giải làm mất vật nhỏ, còn giảm độ dài clip làm mất chính thứ ta đang muốn học. Không có hướng nào miễn phí, và biết trước điều đó tiết kiệm hàng tuần thí nghiệm.

↔ **ràng buộc bởi** $B/11$ — tích độ dài clip $\times$ độ phân giải² $\times$ batch bị chặn bởi bộ nhớ thiết bị — cùng một trần đã chi phối mọi lựa chọn ở chương giới hạn tính toán.

A. Bốn họ biểu diễn, và nút thắt mà mỗi họ gỡ



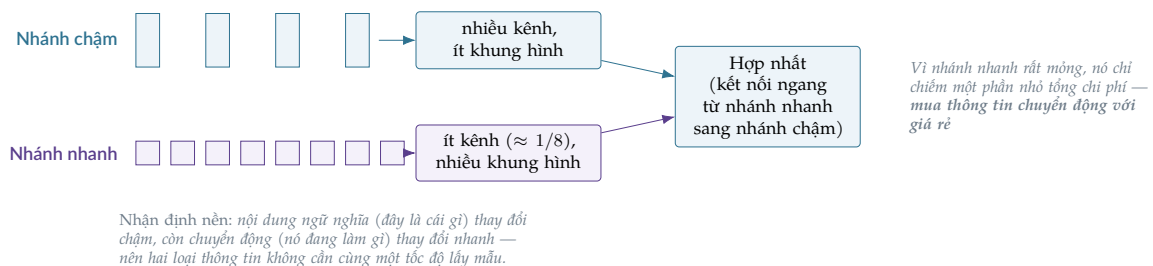
Hình 1.5. Dòng thời gian biểu diễn video, đọc theo nút thắt được gỡ. Nhân tím dưới mỗi mũi tên là *vấn đề của bước trước* — đó là thứ khiến bước sau ra đời. Chú ý rằng đây không phải một thang chất lượng: bước đầu tiên vẫn là baseline mạnh và bắt buộc phải chạy.

Hình 1.5 cho thấy mạch phát triển, còn Bảng 1.3 cho thấy cái mà mạch đó che đi: mỗi họ giả định một điều khác nhau về bản chất của chuyển động, và chính giả định — chứ không phải số tầng — quyết định nó hỏng ở đâu.

Bảng 1.3. Năm cách biểu diễn video. Cột chi phí ghi theo bậc độ lớn tương đối so với một mô hình ảnh chạy trên một khung hình, không phải số đo thực nghiệm.

Cách	Giả định	Chi phí	Được	Hỏng khi nào
Mỗi khung + gộp	Thời gian không quan trọng lắm	$\sim T \times$	Rẻ, tái dùng nguyên mô hình ảnh	Không phân biệt được hành động ngược chiều (mở/đóng, nhặt/đặt)
Hai luồng	Chuyển động tách được thành một nhánh riêng [8]	$\sim 2T \times + \text{flow}$	Rõ ràng, hiệu quả với ít dữ liệu	Phải tính optical flow ngoài mạng; flow sai thì nhánh đó thành nhiễu
3D conv	Không-thời gian đối xứng [9]	$\sim 3T \times$	Học chuyển động đầu-cuối	Tham số và bộ nhớ tăng mạnh; giả định đối xứng vốn sai
(2+1)D	Trục thời gian tách được khỏi trục không gian [10]	$\sim 1,5T \times$	Cùng số tham số nhưng thêm một hàm kích hoạt, dễ tối ưu hơn	Chuyển động thực sự xoắn không-thời gian (xoáy, biến dạng nhanh)
Video transformer	Quan hệ thời gian tùy ý, không cần cục bộ	bậc hai theo số token	Mạnh nhất khi đủ dữ liệu	Chi phí bậc hai theo T ; rất đói dữ liệu

Một họ nữa nằm ngoài bảng vì nó là một ý tưởng chéo chứ không phải một loại tích chập: **Slow-Fast** chạy *hai* nhánh trên cùng video — một nhánh tốc độ khung thấp nhưng nhiều kênh để nắm ngữ nghĩa, một nhánh tốc độ khung cao nhưng ít kênh để nắm chuyển động [11]. Ý nghĩa của thiết kế này: *nội dung ngữ nghĩa thay đổi chậm, còn chuyển động thay đổi nhanh, nên hai loại thông tin không cần cùng một tốc độ lấy mẫu*. Nhánh nhanh rất mỏng: bài báo dùng khoảng một phần tám số kênh. Nó vì thế chỉ chiếm một phần nhỏ tổng chi phí. Đây là ví dụ đẹp của việc dùng cấu trúc bài toán để mua thông tin rẻ hơn.



Hình 1.6. SlowFast dùng cấu trúc của tín hiệu để mua thông tin rẻ hơn. Thay vì tăng đồng loạt tần số lấy mẫu cho toàn mạng — cách tốn kém nhất — nó tách thành hai nhánh có tần số và bề dày kênh ngược nhau, rồi hợp nhất. Nếu nhận định nền sai với dữ liệu của bạn (ngữ nghĩa thay đổi nhanh, chẳng hạn cảnh cắt dựng liên tục) thì lợi thế chi phí biến mất.

Hình 1.6 đáng đặt cạnh Hình 1.7: cả hai đều là ví dụ của cùng một chiến lược — đọc ra một bất đối xứng thật của bài toán rồi đưa nó vào kiến trúc, thay vì trả tiền đồng đều cho mọi chiều.

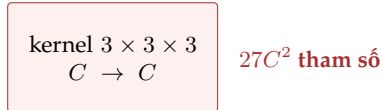
B. Ba phép tính quyết định mọi thứ

- **Bộ nhớ của một tensor kích hoạt.** Một clip 16 khung, 224×224 , 64 kênh, fp32: $16 \times 224 \times 224 \times 64 \times 4 \text{ byte} \approx 205 \text{ MB}$ — cho *một* tensor, ở *batch* bằng 1. Lướt ngược phải giữ kích hoạt

của mọi tầng, nên con số này nhân với số tầng nông. Đây là lý do thực tế vì sao mô hình video hầu như luôn chạy ở batch nhỏ, và vì sao chuẩn hoá theo batch trở thành vấn đề (xem C/12-co-che-huan-luyen/02-lam-cho-training-chay-duoc).

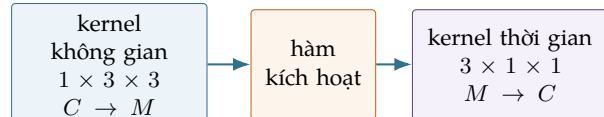
- **Vì sao (2+1)D không tốn thêm tham số.** Một kernel 3D $3 \times 3 \times 3$ từ C kênh sang C kênh có $27C^2$ tham số. Bản tách gồm hai kernel. Kernel không gian $1 \times 3 \times 3$ đi từ C sang M kênh, tốn $9CM$; kernel thời gian $3 \times 1 \times 1$ đi từ M về C , tốn $3MC$. Tổng là $12CM$. Cho hai vế bằng nhau: $M = 2,25C$. *Ý nghĩa:* với cùng ngân sách tham số, ta mua được một hàm kích hoạt bổ sung ở giữa và một bài toán tối ưu dễ hơn — lợi ích đến từ dạng của hàm, không từ sức chứa (capacity) [10].

Tích chập 3D



Một khối duy nhất trộn cả ba trục, ngâm giữ định thời gian đối xứng với không gian

Bản tách (2+1)D



tổng $12CM$; cho bằng $27C^2$ thì $M = 2,25C$

Cùng ngân sách tham số, nhưng mua thêm được một hàm kích hoạt ở giữa — lợi ích đến từ dạng của hàm, không từ sức chứa

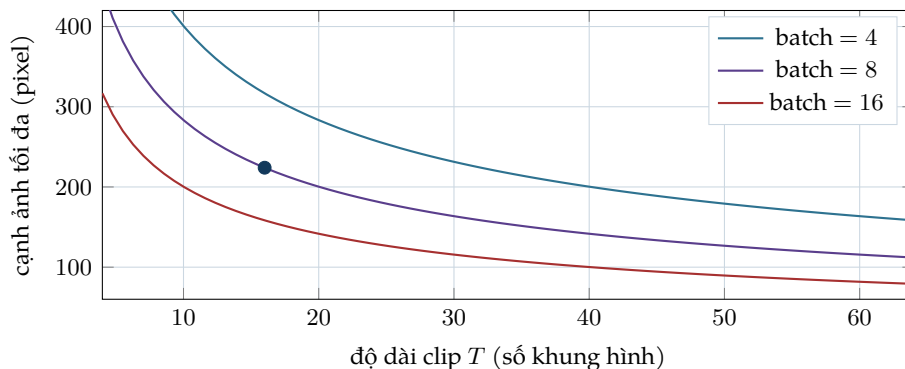
Hình 1.7. Phân rã (2+1)D vẽ theo khối. Kernel 3D duy nhất được tách thành một kernel chỉ tác động theo không gian rồi một kernel chỉ tác động theo thời gian, với số kênh trung gian M chọn sao cho tổng tham số bằng bản gốc. Phép tách phản ánh đúng bất đối xứng của bài toán: trục thời gian có hướng và có đơn vị khác, nên không có lý do để cùng một kernel đối xứng là hợp lý ở cả ba trục.

Hình 1.7 làm rõ vì sao đây không phải mẹo tiết kiệm: nếu chỉ để tiết kiệm thì người ta đã giảm số kênh, chứ không giữ nguyên tham số rồi chèn thêm một phi tuyến.

- **Vì sao attention trên video đắt tới mức đó.** Một ViT chia ảnh 224×224 thành $14 \times 14 = 196$ token, nên ma trận attention có $196^2 \approx 3,8 \times 10^4$ phần tử. Với clip 16 khung, số token thành 3136 và ma trận thành $9,8 \times 10^6$ — gấp 256 lần, đúng bằng 16^2 .

↪ **ràng buộc bởi** C/13/03 — video transformer bỏ thêm một tầng thiên lệch quy nạp nữa — lần này theo trục thời gian — nên nó đòi dữ liệu hơn cả ViT trên ảnh.

Ý nghĩa: chi phí bậc hai theo số token nghĩa là bậc hai theo độ dài clip, nên mọi video transformer thực dụng đều phải phân tách attention (theo thời gian rồi theo không gian) hoặc giới hạn cửa sổ.



Hình 1.8. Đường đẳng ngân sách bộ nhớ. Vì lượng kích hoạt tỉ lệ với $T \cdot R^2 \cdot B$, một ngân sách cố định cho $R \propto 1/\sqrt{TB}$. Điểm xanh đậm là cấu hình tham chiếu (16 khung, 224 px, batch 8). Hình cho thấy điều mà một bảng số không cho thấy: muốn gấp đôi độ dài clip, bạn chỉ phải giảm cạnh ảnh khoảng 30%, nhưng muốn gấp bốn thì phải giảm một nửa — và đường cong dốc rất nhanh ở phía trái.

Hình 1.8 nên dùng như một công cụ chứ không phải một minh họa: khi ai đó đề nghị “tăng độ dài clip lên 64 khung”, hãy chỉ ngay vào đường cong và hỏi họ định trả bằng độ phân giải hay bằng batch.

C. Giới hạn và trường hợp hỏng

- **Rất nhiều tập dữ liệu không thực sự cần thời gian.** Nhiều tập dữ liệu hành động phổ biến có thể đoán đúng phần lớn chỉ từ *bối cảnh* của một khung hình — thấy bể bơi thì đoán “bơi”. Đây là quan sát được lặp lại rộng rãi trong lĩnh vực, và hệ quả là một mô hình thời gian tinh vi có thể trông tốt vì lý do hoàn toàn khác với lý do bạn nghĩ.
- **Optical flow là một phụ thuộc ngoài mạng.** Nhánh chuyển động của kiến trúc hai luồng đòi hỏi flow được tính trước, thường tốn hơn cả mô hình chính. Chất lượng flow khi đó thành trăn cho toàn hệ thống — đúng mẫu “đẩy độ khó sang chỗ khác”.
- **Batch nhỏ phá thống kê chuẩn hoá.**
 ⇔ **hệ quả của** C/12/02 — chuẩn hoá theo batch ước lượng trung bình và phương sai từ chính batch, nên batch nhỏ làm ước lượng nhiễu — đúng cơ chế đã bàn ở phần làm cho huấn luyện chạy được.
 Mô hình video thường chạy batch 2–8 trên mỗi GPU, nên batch norm cho ước lượng phương sai rất nhiễu; đây là lý do các kiến trúc video hay dùng chuẩn hoá theo nhóm hoặc theo tầng [12].
- **Lấy mẫu khung là một siêu tham số ẩn.** Lấy 16 khung liên tiếp, hay 16 khung rải đều trên 4 giây, là hai bài toán khác nhau; và cách lấy mẫu lúc đánh giá thường khác lúc huấn luyện, nên số công bố phụ thuộc vào một chi tiết ít khi được nêu rõ.

CẠM BẮY

Trước khi so sánh bất kỳ hai kiến trúc video nào, hãy chạy baseline rẻ nhất: lấy một khung hình ngẫu nhiên, chạy mô hình ảnh, rồi trung bình dự đoán trên vài khung. Nếu baseline này đạt gần bằng mô hình thời gian, thì tập dữ liệu của bạn không đo cái bạn tưởng nó đo, và mọi so sánh giữa các mô hình thời gian phía trên đều là nhiễu. Đây là bài kiểm tra rẻ nhất và bị bỏ qua nhiều nhất trong cả lĩnh vực.

CẦN NHỚ

Ngân sách bộ nhớ của video chia cho ba chiều nhân với nhau: độ dài clip $\times$ độ phân giải² $\times$ batch. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Và trước khi trả tiền cho chiều thời gian, hãy chứng minh rằng bài toán của bạn thật sự cần nó, bằng baseline một khung hình. [T2]

D. Kết nối

B/11-gioi-han-tinh-toan là nơi ngân sách bộ nhớ và băng thông được xử lý tổng quát; mục này chỉ là trường hợp riêng nhưng khác nghiệt nhất. C/13-kien-truc-backbone/03-vit-va-inductive-bias giải thích vì sao video transformer đối dữ liệu: bỏ inductive bias theo trục thời gian còn tốn dữ liệu hơn bỏ theo trục không gian. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc tiếp nối trực tiếp bằng nhánh optical flow — thứ mà kiến trúc hai luồng dựa vào — và bằng ràng buộc nhân quả, thứ loại bỏ phần lớn các kiến trúc trong bảng trên khi bạn phải chạy streaming.

❏ **Câu hỏi tự kiểm** (trả lời không nhìn bài)

1. Vì sao coi thời gian như trục không gian thứ ba là một giả định sai, và sai đó biểu hiện thành lỗi gì trong thực tế?
2. Tính lượng bộ nhớ cho một tensor kích hoạt clip 32 khung, 112×112 , 128 kênh, fp16. Nếu vượt ngân sách, bạn cắt chiều nào trước và vì sao?
3. (2+1)D có cùng số tham số với 3D conv nhưng thường tốt hơn. Lợi ích đó đến từ đâu, nếu không phải từ sức chứa của mô hình?
4. Vì sao chi phí attention trên clip 16 khung gấp 256 lần trên một ảnh, chứ không phải 16 lần?
5. SlowFast dựa trên nhận định nào về bản chất của tín hiệu? Nếu nhận định đó sai với dữ liệu của bạn thì kiến trúc hồng thể nào?
6. Baseline “một khung hình rồi gộp” đạt 92% so với mô hình 3D đạt 94%. Bạn kết luận gì, và bước tiếp theo là gì?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [5] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [7] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [8] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [9] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [10] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [11] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [12] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [13] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [14] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [15] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [16] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [17] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [18] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [19] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.

1.3 Flow, tracking và các ràng buộc riêng (Optical Flow, Tracking and Their Constraints)

PHỤ THUỘC C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video (ngân sách bộ nhớ); B/07-chia-du-lieu-va-danh-gia (rõ ràng dữ liệu, chọn metric); A/01-toan-toi-thieu (bình phương tối thiểu, điều kiện của ma trận).

TÓM TẮT

Mục này bàn ba thứ mà một kỹ sư SLAM đã sống cùng hằng ngày, chỉ dưới tên khác: **dòng quang** (*optical flow*) là bài toán tương ứng điểm, **bám nhiều mục tiêu** (*multi-object tracking*) là bài toán liên kết dữ liệu, và ràng buộc nhân quả là ràng buộc cứng của mọi hệ chạy thời gian thực. Đọc xong bạn ánh xạ được các kỹ thuật học sâu ở đây về đúng chỗ tương ứng trong pipeline hình học của mình. Bạn chọn được metric tracking không đánh lừa. Và bạn biết kiến trúc nào bị loại ngay từ đầu khi không được nhìn khung hình tương lai. Nếu chỉ nhớ một điều: trong cả flow lẫn tracking, chỗ hệ thống sống hay chết là bước *liên kết* chứ không phải bước phát hiện — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng chỗ đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **optical flow** — trường vector (u, v) cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. “Dày đặc” nghĩa là tính cho mọi pixel, khác với việc chỉ bám vài trăm đặc trưng.
- **gradient ảnh** I_x, I_y, I_t — tốc độ thay đổi độ sáng theo trục ngang, trục dọc và theo thời gian tại một pixel. Ba đại lượng này là toàn tập dữ liệu mà phương trình flow cổ điển có.
- **bài toán khẩu độ (aperture problem)** — khi chỉ nhìn qua một cửa sổ nhỏ, chỉ thành phần chuyển động vuông góc với cạnh là quan sát được; thành phần dọc theo cạnh hoàn toàn vô hình. Đây là thiếu thông tin thật, không phải lỗi thuật toán.
- **liên kết dữ liệu (data association)** — bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking đó là ghép hộp với track; trong SLAM đó là ghép đặc trưng với điểm bản đồ. Gán sai gây hậu quả nặng và khó rút lại.
- **track ID** — mã số mà hệ thống gán cho một đối tượng và phải giữ nguyên suốt thời gian nó xuất hiện. *ID switch* là khi mã số bị đánh tráo giữa hai đối tượng — lỗi liên kết điển hình.
- **bộ lọc Kalman** — bộ ước lượng đệ quy giữ vị trí và vận tốc cùng độ bất định của chúng; nó dự đoán vị trí khung tới rồi cập nhật khi có phép đo mới. Trong tracking, nó cung cấp dự đoán để ghép cặp.
- **thuật toán Hungary** — thuật toán giải bài toán gán tối ưu: cho ma trận chi phí giữa m dự đoán và n phát hiện, tìm phép ghép một-một có tổng chi phí nhỏ nhất.
- **MOTA và HOTA** — hai thước đo tracking. MOTA cộng số phát hiện sót, phát hiện thừa và số lần đổi mã rồi chia cho tổng số vật thật. HOTA tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA) rồi lấy trung bình nhân của chúng.
- **nhân quả / streaming** — ràng buộc rằng quyết định tại thời điểm t chỉ được dùng dữ liệu tới t , không được dùng khung hình tương lai. Nó loại bỏ mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước.

Đặt vấn đề

Hai mục trước xử lý video như một khối cần phân loại. Nhưng phần lớn ứng dụng thật lại cần thứ tinh hơn nhiều: *cùng một điểm vật lý ở khung này nằm ở đâu trong khung kia*. Trả lời câu đó cho mọi pixel là optical flow; trả lời cho từng vật thể qua thời gian là tracking; và trong cả hai trường hợp, khó khăn thật không nằm ở việc nhìn thấy mà ở việc *ghép cặp*. Đây chính xác là cấu trúc của bài toán mà một hệ SLAM giải mỗi khung hình, nên mục này ít giới thiệu khái niệm mới hơn là dịch từ vựng.

Thêm vào đó là một ràng buộc mà ảnh tĩnh không có: mô hình chạy trực tuyến không được nhìn khung hình tương lai, và cũng không được nhìn quá lâu vào quá khứ nếu ngân sách mili-giây

có hạn. Ràng buộc này loại bỏ nguyên một lớp kiến trúc trước khi độ chính xác kịp lên tiếng — đúng nghĩa **ràng buộc cứng** ở B/03-giai-phau-he-thong.

Học xong mục này bạn làm được gì mà trước đó không làm được? Bạn đọc một paper flow hay tracking và biết ngay nó thay khối nào trong pipeline hình học quen thuộc, nó mua độ bền bằng giả định gì, và nó có chạy được dưới ràng buộc nhân quả của bạn hay không.

TRỰC GIÁC

Nhìn một cạnh thẳng đang chuyển động qua một cái ống hút: bạn chỉ thấy được thành phần chuyển động *vuông góc* với cạnh, còn thành phần dọc theo cạnh thì hoàn toàn vô hình. Đó không phải hạn chế của mắt hay của thuật toán, mà là thiếu thông tin thật sự: một phương trình, hai ẩn. Mọi thứ trong optical flow — lấy cửa sổ, làm trơn, kim tự tháp, khối tương quan — đều là các cách khác nhau để mượn thêm ràng buộc từ nơi khác. Và ma trận đo mức độ “mượn được hay không” chính là ma trận mà bạn đã dùng để chọn đặc trưng góc trong SLAM.

A. Optical flow: từ Lucas–Kanade tới RAFT

Giả định gốc là **độ sáng không đổi** (*brightness constancy*): cùng một điểm vật lý giữ nguyên giá trị sáng giữa hai khung. Viết $I(x, y, t) = I(x + u, y + v, t + 1)$ rồi khai triển Taylor bậc nhất cho phương trình ràng buộc

$$I_x u + I_y v + I_t = 0.$$

Câu này nói gì bằng lời: *tốc độ thay đổi độ sáng theo thời gian tại một pixel phải bằng đúng phần độ sáng bị “trôi vào” do chuyển động*. Một phương trình, hai ẩn — đó là **bài toán khâu độ** (*aperture problem*) ở dạng đại số.

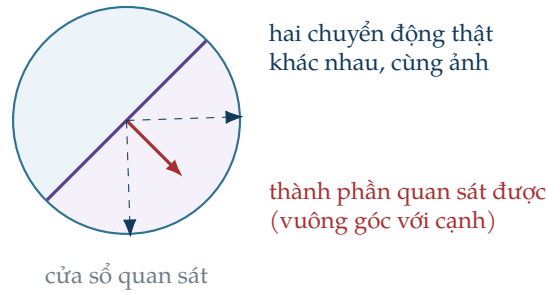
Lucas–Kanade mượn ràng buộc bằng cách giả định flow không đổi trong một cửa sổ nhỏ, cho N phương trình và hai ẩn, rồi giải bình phương tối thiểu. Phương trình chuẩn có ma trận hệ số

$$A^T A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix},$$

và đây đúng là ma trận cấu trúc trong tiêu chuẩn chọn đặc trưng góc. Ý nghĩa: bài toán giải được khi và chỉ khi cửa sổ có gradient theo *hai* hướng độc lập. Ví dụ nhỏ nhất: một cửa sổ nằm trên một cạnh dọc lý tưởng có $I_x = g$ và $I_y = 0$ ở mọi pixel, nên $A^T A = \begin{bmatrix} Ng^2 & 0 \\ 0 & 0 \end{bmatrix}$ — suy biến, giá trị riêng nhỏ nhất bằng 0, và thành phần flow dọc theo cạnh không xác định.

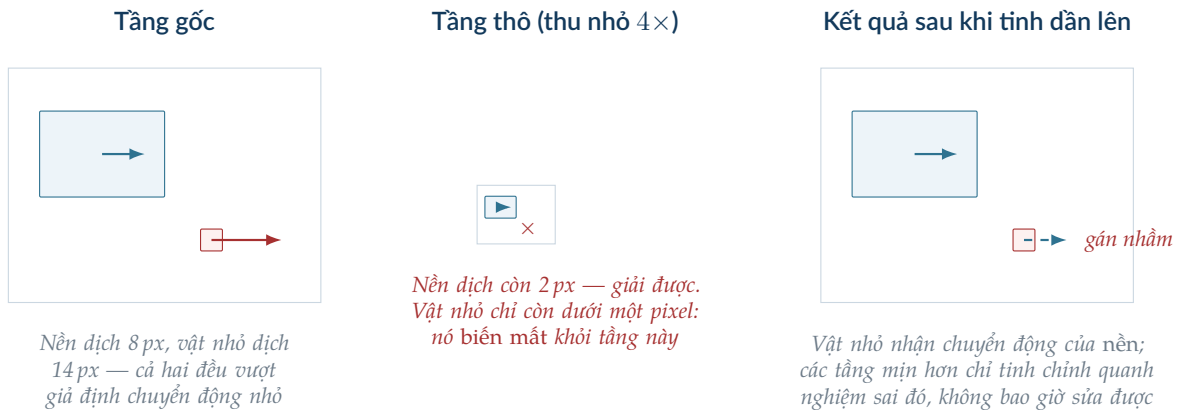
↪ **tiền đề** A/01 — đây đúng là câu hỏi điều kiện của một hệ bình phương tối thiểu: nghiệm ổn định khi giá trị riêng nhỏ nhất đủ xa 0, và suy biến khi nó bằng 0.

Chọn “good features to track” chính là chọn các cửa sổ mà giá trị riêng nhỏ nhất của ma trận này đủ lớn; nói cách khác, việc chọn đặc trưng trong SLAM và tính giải được của flow là *cùng một điều kiện*.



Hình 1.9. Bài toán khâu độ. Hai chuyển động thật rất khác nhau (mũi tên đứt) tạo ra cùng một hình ảnh bên trong cửa sổ, vì chỉ thành phần vuông góc với cạnh là quan sát được. Về đại số, đó là việc ma trận $A^T A$ suy biến; về thực hành, đó là lý do mọi bộ bám đặc trưng đều phải chọn góc chứ không chọn cạnh.

Hình 1.9 cũng giải thích luôn giả định thứ ba của Lucas–Kanade — *chuyển động nhỏ* — vì khai triển Taylor bậc nhất chỉ đúng khi dịch chuyển nhỏ hơn cỡ một pixel gradient. Cách chữa cổ điển là kim tự tháp thô–tối–mịn: giảm ảnh xuống bốn lần thì chuyển động 8 pixel thành 2 pixel. Cái giá là một chế độ hỏng đặc trưng: vật *nhỏ* và *nhạy* biến mất ở tầng thô, nên hệ thống ước lượng chuyển động của nền thay vì của vật, và sai lầm đó không bao giờ được sửa ở các tầng mịn hơn.



Hình 1.10. Chế độ hỏng đặc trưng của chiến lược thô–tối–mịn. Thu nhỏ ảnh làm chuyển động lớn trở nên nhỏ — đó là mục đích — nhưng nó cũng làm vật nhỏ biến mất trước khi kịp được ước lượng. Vì mỗi tầng chỉ *tinh chỉnh* nghiệm của tầng thô hơn, một sai lầm ở tầng thô là sai lầm vĩnh viễn. Đây chính là lý do RAFT bỏ kim tự tháp và tính tương quan toàn cặp một lần.

Hình 1.10 nên đọc như một cảnh báo quen thuộc với người làm bám đặc trưng: mọi sơ đồ nhiều tầng chỉ tinh chỉnh chứ không sửa lại đều thừa hưởng đúng chế độ hỏng này, kể cả bộ tối ưu bạn tự viết.

Bảng 1.4. Bốn thể hệ optical flow. Cột cuối là cột đáng giá nhất: mỗi phương pháp mượn ràng buộc từ một nguồn khác nhau, nên nó hỏng khi đúng nguồn đó cạn.

Phương pháp	Mượn ràng buộc từ đâu	Mạnh khi nào	Hỏng khi nào
Lucas–Kanade	Giả định flow hằng trong cửa sổ nhỏ	Chuyển động nhỏ, kết cấu giàu, rất rẻ	Vùng trơn, cạnh dài, chuyển động lớn; thay đổi độ sáng phá giả định gốc
LK + kim tự tháp	Thêm giả định trơn qua các tỉ lệ	Chuyển động lớn của vùng lớn	Vật nhỏ chuyển động nhanh biến mất ở tầng thô
FlowNet, PWC-Net	Học từ dữ liệu; PWC-Net đưa kim tự tháp, warping, khối chi phí thành kiến trúc [13], [14]	Cân bằng tốt giữa chính xác và tốc độ	Lệch miễn so với dữ liệu tổng hợp dùng để huấn luyện
RAFT	Khối tương quan <i>toàn cặp</i> + tinh chỉnh lặp bằng GRU [15]	Chuyển động lớn, biên sắc, khái quát tốt	Bộ nhớ của khối tương quan; số vòng lặp là ngân sách phải tự chọn

RAFT đáng chú ý riêng vì hai lý do rất gần với kinh nghiệm tối ưu phi tuyến. Thứ nhất, nó bỏ chiến lược thô–tối–mịn bằng cách tính tương quan giữa *mọi* cặp pixel một lần, nhờ đó chuyển động lớn không còn cần kim tự tháp. Cái giá đo được: ở độ phân giải $1/8$ của ảnh 1024×436 , lưới là $128 \times 54 \approx 6900$ điểm, nên khối toàn cặp có $6900^2 \approx 4,8 \times 10^7$ phần tử — khoảng 190 MB ở fp32, trước khi tính bất cứ thứ gì. Thứ hai, nó tinh chỉnh nghiệm bằng một số vòng lặp cố định thay vì giải một lần; điều này khiến *số vòng lặp trở thành núm vận độ trễ*, đúng như việc bạn giới hạn số bước Gauss–Newton trong một bộ tối ưu thời gian thực. Đây là chỗ nổi đáng nhớ: mô hình học sâu ở đây không thay thế tư duy tối ưu lặp, nó mượn lại tư duy đó.

B. Nhãn nằm trên khoảng thời gian, không nằm trên khung hình

Nhận dạng hành động trên clip đã cắt sẵn là bài toán dễ; *temporal action localization* mới là bài toán thật: cho một video dài, tìm *khoảng* nào chứa hành động gì. Việc nhãn nằm trên khoảng kéo theo hai hệ quả mà người quen bài toán ảnh dễ vấp.

- **Metric phải đổi.** Ta không đánh giá theo khung hình mà theo khoảng, với ngưỡng *temporal IoU*. Ranh giới hành động vốn mơ hồ ngay cả với người gán nhãn — “bắt đầu ngồi xuống” là khung nào? — nên độ chính xác ở ngưỡng tIoU chặt phản ánh sự bất đồng của người gán nhãn nhiều hơn phản ánh chất lượng mô hình.
- **Chia dữ liệu phải theo video, không theo clip.** Hai clip cắt từ cùng một video có nền, ánh sáng và diễn viên giống nhau; để chúng nằm hai bên tập train/test là rò rỉ dữ liệu kinh điển, và nó thổi phồng kết quả một cách rất khó phát hiện.

↔ **trường hợp riêng của B/07** — hai clip cắt từ cùng một video là một nhóm phụ thuộc; chia ngẫu nhiên theo clip là rò rỉ dữ liệu kinh điển, chỉ đổi trục từ không gian sang thời gian.

C. Tracking chính là liên kết dữ liệu

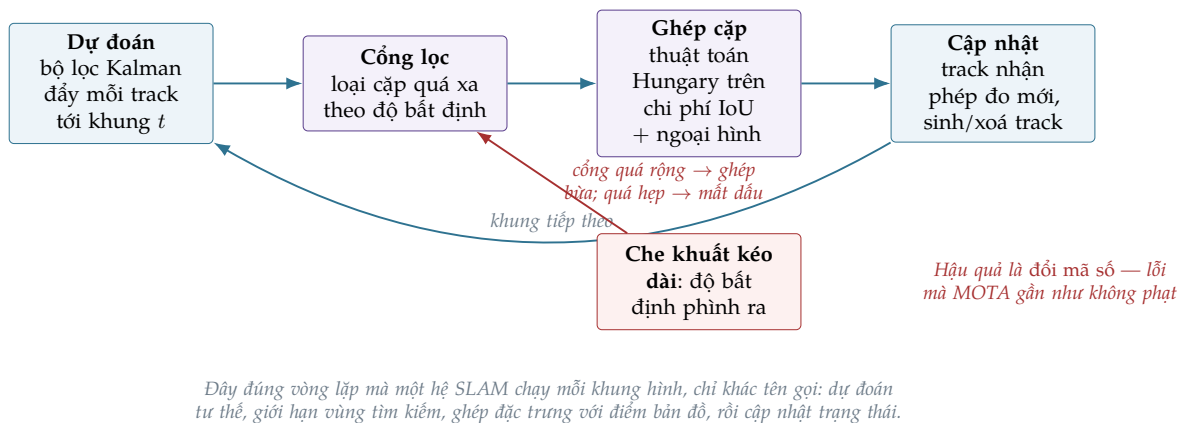
Có hai triết lý. *Tracking-by-detection* tách hẳn hai việc: một detector chạy độc lập trên mỗi khung, rồi một khối liên kết nối các phát hiện qua thời gian bằng một mô hình chuyển động cộng một độ đo tương đồng. *Joint detection-tracking* huấn luyện cả hai cùng lúc, mạnh hơn trên giấy nhưng khó gỡ lỗi vì không còn ranh giới rõ để đặt điểm dừng. Với hệ thống sản xuất, cách thứ nhất

thường thắng vì lý do kỹ thuật hệ thống chứ không phải vì độ chính xác: bạn thay được detector mà không phải huấn luyện lại phần liên kết.

Khối liên kết là chỗ mà người làm SLAM sẽ thấy quen tới mức bất ngờ. SORT dùng đúng bộ đôi kinh điển — bộ lọc Kalman dự đoán vị trí khung tới, rồi thuật toán Hungary ghép cặp dự đoán với phát hiện theo chi phí IoU [16]. ByteTrack thêm một quan sát đơn giản mà hiệu quả: đừng vứt các phát hiện điểm thấp, hãy dùng chúng ở vòng ghép thứ hai, vì một vật bị che một phần thường cho điểm thấp chứ không phải không có [17]. Về mặt cấu trúc, đây là cùng bài toán **liên kết dữ liệu** (*data association*) mà một hệ SLAM giải khi ghép đặc trưng với điểm bản đồ: có dự đoán từ mô hình chuyển động, có cổng lọc theo khoảng cách thống kê, có một bài toán gán tối ưu, và có hậu quả nghiêm trọng khi gán sai.

→ cùng nguyên lý với C/16 — ghép hộp với track và ghép đặc trưng với điểm bản đồ là cùng một bài toán gán tối ưu có cổng lọc theo độ bất định; chỉ khác hậu quả khi gán sai.

Khác biệt lớn nhất là ở SLAM, một liên kết sai làm hỏng bản đồ và rất khó rút lại, trong khi ở tracking nó chỉ đổi một mã số — nhưng chính vì hậu quả nhẹ hơn nên metric mới dễ che giấu nó.



Hình 1.11. Tracking là một vòng lặp liên kết dữ liệu, không phải một mạng nơ-ron. Ba trong bốn khối không chứa tham số học được nào; phần học được nằm ở detector cấp phép đo và ở đặc trưng ngoại hình dùng làm chi phí ghép. Vì thế cải thiện detector và cải thiện liên kết là hai việc khác nhau, và một thước đo gộp chúng vào một số sẽ che mất việc nào đang hỏng.

Hình 1.11 là lý do hộp cạm bẫy ngay dưới đây quan trọng đến vậy: nếu vòng lặp này hỏng mà thước đo không phản ánh, bạn sẽ dành công sức tối ưu nhầm khối.

CẠM BÃY

MOTA cộng ba loại lỗi rồi chia cho tổng số vật thể thật: $MOTA = 1 - (FN + FP + IDSW) / GT$. Vấn đề nằm ở việc FN và FP được đếm *trên từng khung hình cho từng vật*, còn mỗi lần đổi mã số chỉ được đếm *một lần*. Ví dụ: 1000 vật–khung, 50 FN, 30 FP, 40 lần đổi mã cho $MOTA = 0,88$; cải thiện khối liên kết để chỉ còn 20 lần đổi mã — tức tốt lên gấp đôi — chỉ nâng MOTA lên 0,90. Nói cách khác, một hệ thống liên tục đánh tráo danh tính vẫn có thể khoe MOTA cao. HOTA sửa điều này bằng cách tách riêng chất lượng phát hiện và chất lượng liên kết, rồi lấy trung bình nhân $\sqrt{DetA \cdot AssA}$. Hai thành phần khi đó có trọng số ngang nhau, và không thành phần nào bù đắp được cho thành phần kia [18], [19].

D. Nhất quán thời gian và ràng buộc nhân quả

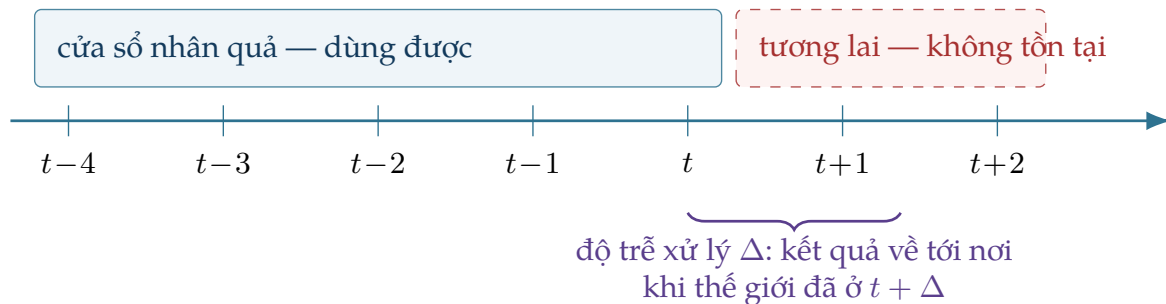
Một mô hình phân vùng chạy độc lập trên từng khung có thể đạt mIoU cao mà vẫn cho video nhấp nháy. Lý do là hàm mất mát không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau. Ba cách xử lý, với ba mức chi phí khác nhau:

Bảng 1.5. Ba cách chữa hiện tượng nhấp nháy, xếp theo mức xâm lấn vào hệ thống.

Cách	Cơ chế	Chi phí	Hỏng khi nào
Làm mượt hậu xử lý	Lọc dự đoán qua vài khung, có thể warp bằng flow	Rẻ, nhưng thêm độ trễ	Chuyển cảnh và chuyển động nhanh: làm mượt thành làm nhoè
Đưa thời gian vào mô hình	Mô hình nhận nhiều khung, hoặc mang trạng thái ẩn	Đắt nhất, chạm vào ngân sách ở Mục 2	Trạng thái ẩn có thể “dính” vào một sai lầm và tự củng cố
Loss nhất quán thời gian	Phạt sai khác giữa dự đoán của hai khung liên tiếp sau khi warp	Trung dung, chỉ tốn lúc huấn luyện	Warp dựa vào flow, nên flow sai thì loss dạy điều sai

↔ **ràng buộc bởi C/15/02** — cách chữa “đưa thời gian vào mô hình” nghe hấp dẫn nhất nhưng chính là cách đắt nhất, vì nó tiêu vào đúng ngân sách ba chiều đã cạn ở mục trước.

Cả ba đều bị một ràng buộc lớn hơn chi phối: hệ thống chạy trực tuyến *không được nhìn khung hình tương lai*. Ràng buộc này loại thẳng mọi kiến trúc hai chiều, mọi phép làm mượt trung tâm, và mọi lịch trình đánh giá “xem toàn bộ video rồi mới quyết định” — và nó loại chúng trước khi độ chính xác được đem ra bàn.



Hình 1.12. Ràng buộc nhân quả, vẽ trên trục thời gian. Vùng xanh là toàn bộ thông tin mà một hệ streaming được phép dùng để quyết định tại thời điểm t ; vùng đỏ là thứ mà mọi mô hình hai chiều lạng lẽ giả định là có. Ngoặc tím nhắc thêm một điều thường bị quên: ngay cả quyết định đúng cũng đến muộn mất Δ , nên hệ thống hạ nguồn phải hoặc dự đoán bù, hoặc chấp nhận hành động theo một trạng thái đã cũ.

Hình 1.12 nói thêm một điều mà người làm robot cảm nhận rõ hơn ai hết: ràng buộc nhân quả không chỉ là “không được nhìn tương lai” mà còn là “kết quả luôn nói về quá khứ”. Vì thế, khi so sánh hai mô hình cho hệ thời gian thực, đại lượng đáng so không phải độ chính xác trên khung hình t . Đáng so là độ chính xác *tại thời điểm quyết định được dùng*. Cách đánh giá này vẫn còn ít được chuẩn hoá, và nhiều bảng kết quả công bố không nói gì về nó.

E. Giới hạn và trường hợp hỏng

- **Giả định độ sáng không đổi bị vi phạm thường xuyên hơn ta nghĩ.** Tự động phơi sáng, đèn nhấp nháy, phản xạ gương và bề mặt bóng đều làm sai giả định gốc của flow — và đây đúng là những cảnh làm hỏng cả các hệ thống hình học dựa trên độ sáng. Mô hình học sâu chịu

đựng tốt hơn nhưng không miễn nhiễm, vì dữ liệu huấn luyện của chúng phần lớn là ảnh tổng hợp phối sáng cố định.

- **Che khuất kéo dài là kẻ thù của mọi bộ bám.** Mô hình chuyển động dự đoán được vài khung, sau đó độ bất định lớn tới mức cổng lọc hoặc quá rộng (ghép bù) hoặc quá hẹp (mất dấu). Không có cách chỉnh tham số nào thoát khỏi đánh đổi này; chỉ có thêm thông tin — ngoại hình, hoặc một cảm biến khác.
- **Metric đánh lừa theo hai kiểu.** MOTA che giấu lỗi liên kết như đã nói; còn mIoU theo khung che giấu hiện tượng nhấp nháy. Cả hai đều là ví dụ của cùng một sai lầm: chọn một con số tổng không đo được đại lượng mà hệ thống hạ nguồn thực sự cần.
- **Lệch miền của dữ liệu flow.** Vì nhãn flow dày đặc gần như không thể gán bằng tay, các mô hình được huấn luyện chủ yếu trên dữ liệu tổng hợp; chất lượng trên cảnh thật vì thế phụ thuộc vào mức độ giống nhau giữa hai miền, và điều đó thay đổi theo camera của bạn.

CẦN NHỚ

Trong flow cũng như trong tracking, phần khó không phải phát hiện mà là *liên kết* — và cả hai lĩnh vực đều có một metric phổ biến che giấu đúng phần đó. Hãy chọn metric tách riêng chất lượng phát hiện khỏi chất lượng liên kết (HOTA cho tracking, sai số theo từng chế độ chuyển động cho flow), rồi mới chọn mô hình. Ràng buộc nhân quả thì đứng trên tất cả: nó loại phương án trước khi độ chính xác kịp lên tiếng. [T2]

F. Kết nối

[C/15-thi-giac-khong-thoi-gian/02-bieu-dien-video](#) là tiền đề về chi phí: mọi cách chữa nhấp nháy bằng “đưa thời gian vào mô hình” đều phải trả bằng ngân sách ba chiều ở đó. [B/07-chia-du-lieu-va-danh-gia](#) là nơi bàn tổng quát về việc chọn metric và về rõ ràng do nhóm phụ thuộc, mà chia dữ liệu theo clip là một ca điển hình. [C/16-thi-giac-3d](#) nối tiếp trực tiếp: flow là bài toán tương ứng hai chiều, còn chương 16 nâng nó lên tương ứng ba chiều và ràng buộc epipolar. [C/14-bai-toan-cv-loi/04-bai-toan-lan-can](#) chứa cấu trúc đánh đổi song song: top-down và bottom-up ở pose lặp lại đúng dạng của tracking-by-detection và joint tracking.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một phương trình độ sáng không đủ để giải ra flow, và ba giả định của Lucas–Kanade mượn ràng buộc còn thiếu từ đâu?
2. Viết ma trận $A^T A$ cho một cửa sổ nằm trên cạnh dọc lý tưởng và giải thích vì sao nó suy biến. Điều đó liên hệ thế nào với việc chọn đặc trưng góc?
3. Kim tự tháp thô–tới–mịn chứa được chuyển động lớn nhưng sinh ra chế độ hỏng nào, và nó biểu hiện ra sao trong một cảnh giao thông cụ thể?
4. RAFT bỏ kim tự tháp bằng khối tương quan toàn cặp. Cái giá tính được bằng con số là gì, và số vòng lặp đóng vai trò gì với một hệ thời gian thực?
5. Một hệ thống có MOTA 0,90 nhưng đổi mã số liên tục. Giải thích bằng công thức vì sao điều đó có thể xảy ra, và HOTA sửa nó thế nào?
6. Ràng buộc nhân quả loại bỏ những loại kiến trúc và loại đánh giá nào? Vì sao độ trễ Δ khiến “độ chính xác trên khung t ” là một đại lượng không đủ?
7. Bạn có mô hình phân vùng mIoU cao nhưng video nhấp nháy, và ngân sách tính toán đã cạn. Bạn chọn cách chữa nào trong ba cách, và chấp nhận hỏng ở đâu?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [5] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [7] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [8] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [9] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [10] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [11] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [12] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [13] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [14] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [15] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [16] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [17] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [18] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [19] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.

1.4 Thực hành

Chương này có một đặc điểm may mắn: mọi khẳng định trong đó đều kiểm chứng được trên một video 60 giây và một GPU tầm trung. Hãy tận dụng điều đó. Và hãy tự tay làm hỏng hệ thống: các bài dưới đây được thiết kế để bạn quan sát *metric nào tụt trước*. Đó mới là thứ dạy bạn về metric.

Repo và tài nguyên. Tracking: [ifzhang/ByteTrack](#) (đơn giản, mạnh, đọc được trong một buổi), [NirAharon/BoT-SORT](#), và [mikel-brostrom/boxmot](#) nếu muốn thử nhiều bộ liên kết trên cùng một detector. Phân vùng và bám trong video: SAM 3. Bám điểm dày: [facebookresearch/co-tracker](#). Optical flow: [princeton-vl/RAFT](#) và [autonomousvision/unimatch](#). Nhận dạng hành động: [open-mmlab/mmdetection](#), VideoMAE-V2, InternVideo. Video qua mô hình ngôn ngữ-thị giác: Qwen3-VL (định vị trong video dài), Molmo2 (mở cả dữ liệu, theo dõi đối tượng qua khung hình). Đánh giá: [JonathonLuiten/TrackEval](#) — đây là thứ đáng đầu tư thời gian nhất trong danh sách, vì nó tính HOTA đúng cách và tách được lỗi phát hiện khỏi lỗi liên kết. Danh sách này chốt tại tháng 9/2026 và các tên mô hình dẫn đầu sẽ cũ trong 6–12 tháng, trong khi hạ tầng đánh giá như TrackEval thì bền hơn nhiều.

Câu hỏi kiểm tra hiểu. Vì sao MOTA có thể cao trong khi hệ thống liên tục đổi mã số đối tượng, và HOTA sửa điều đó bằng cơ chế nào? Một mô hình phân vùng đạt mIoU 0,85 nhưng video nhấp nháy — ba cách xử lý là gì, và mỗi cách trả giá bằng gì? Nếu bạn phải chạy trực tuyến với ngân sách 30 ms mỗi khung hình, những họ kiến trúc nào trong chương này bị loại ngay từ đầu, và vì lý do gì?

Mini project (vài giờ đến hai ngày). Ghép một detector với ByteTrack trên một video 60 giây, đánh giá bằng TrackEval để có cả MOTA lẫn HOTA. Sau đó *cố tình làm hỏng* detector bằng cách hạ dần ngưỡng tin cậy, và ghi lại metric nào tụt trước, tụt bao nhiêu. Kết thúc bằng một đoạn giải thích vì sao thứ tự tụt lại như vậy — đó là bài học thật của bài tập này.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Nghẽn cổ chai thông tin	Chỗ mà hệ thống nén dữ liệu có cấu trúc thành một vector cố định <i>trước khi</i> biết sẽ cần phần nào. Trong sinh mô tả ảnh, đó là việc nén lưới $14 \times 14 \times 512$ thành 512 số, tỉ lệ 196:1, xoá sạch thông tin “đặc trưng này nằm ở đâu”.
●●● Attention	Phép trung bình có trọng số trên các phần của nguồn, với trọng số phụ thuộc vào thứ đang cần. Nó vẫn nén thành một vector, nhưng nén lại một lần cho mỗi bước đầu ra. Gộp trung bình toàn cục là trường hợp riêng khi mọi trọng số bằng nhau.
●● Vì sao gộp trung bình toàn cục làm hỏng sinh mô tả ảnh?	Vì nó lấy trung bình theo không gian, nên xoá thông tin vị trí. Câu “con chó bên trái người đàn ông” cần đúng thông tin đó. Triệu chứng là mô hình bịa chi tiết hợp lý thay vì mô tả cái có thật.
●●● Kích hoạt	Tensor giá trị trung gian mà mỗi tầng sinh ra, phải giữ trong bộ nhớ vì lượt ngược cần tới. Một clip 16 khung $224 \times 224 \times 64$ ở fp32 chiếm khoảng 205 MB cho <i>một</i> tensor ở batch 1. Đây mới là thứ làm tràn bộ nhớ, không phải số trọng số.
●●● Ngân sách bộ nhớ	Lượng kích hoạt tỉ lệ với độ dài clip $\times$ độ phân giải ² $\times$ batch. Vì tích của ba đại lượng bị phần cứng chặn, mọi lựa chọn kiến trúc video thực chất là chọn hy sinh chiều nào.
●● (2+1)D	Tách kernel 3D thành một kernel chỉ theo không gian rồi một kernel chỉ theo thời gian. Chọn số kênh trung gian $M = 2,25 C$ thì tổng tham số bằng bản 3D. Lợi ích đến từ một hàm kích hoạt thêm vào giữa, không từ sức chứa.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● SlowFast	Hai nhánh chạy trên cùng video: nhánh chậm ít khung nhưng nhiều kênh để nắm ngữ nghĩa, nhánh nhanh nhiều khung nhưng ít kênh để nắm chuyển động. Nhận định nền là ngữ nghĩa đổi chậm còn chuyển động đổi nhanh; nếu nhận định đó sai thì lợi thế chi phí biến mất.
●● Vì sao chi phí video transformer là bậc hai theo độ dài clip?	Vì attention tính quan hệ giữa mọi cặp token, và số token tỉ lệ với số khung. Một ảnh có 196 token cho ma trận $3,8 \times 10^4$ ô; clip 16 khung có 3136 token cho $9,8 \times 10^6$ ô, gấp 256 lần.
●● Mô hình 3D chỉ hơn baseline một khung hình 2 điểm — nghĩa là gì?	Nhiều khả năng tập dữ liệu đoán đúng được từ bối cảnh một khung, tức nó không đo thông tin thời gian. Phân biệt bằng cách chạy thêm trên dữ liệu cần thứ tự, ví dụ phân biệt mở với đóng; nếu baseline sập ở đó thì phép đo mới có nghĩa.
●●● Dòng quang	Trường vector cho biết mỗi pixel của khung này nằm ở đâu trong khung kia. Giả định gốc là độ sáng không đổi, nên nó hồng khi tự động phơi sáng, đèn nhấp nháy hay bề mặt bóng làm giá trị sáng đổi.

●●● phải thuộc ●● nên nắm ● biết là có

Active recall – phần 2 (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Bài toán khẩu độ	Một phương trình độ sáng nhưng hai ẩn, nên qua một cửa sổ nhỏ chỉ đo được thành phần chuyển động vuông góc với cạnh. Đây là thiếu thông tin thật, không phải lỗi thuật toán, nên mọi cách chữa đều là mượn thêm ràng buộc từ nơi khác.
●● Vì sao phải chọn góc chứ không chọn cạnh để bấm?	Vì nghiệm bình phương tối thiểu của Lucas-Kanade cần ma trận $A^T A$ không suy biến, tức cửa sổ phải có gradient theo hai hướng độc lập. Trên một cạnh lý tưởng, giá trị riêng nhỏ nhất bằng 0 và flow dọc cạnh không xác định.
●● RAFT	Bộ ước lượng dòng quang dùng khối tương quan toàn cặp cộng một số vòng tinh chỉnh cố định bằng GRU. Nó bỏ được kim tự tháp nên xử lý tốt chuyển động lớn; giá là khối tương quan cỡ 190 MB ở 1/8 độ phân giải, và số vòng lặp thành nùm vận độ trễ.
●● Vì sao kim tự tháp thô-tối-mịn làm mất vật nhỏ chuyển động nhanh?	Vì thu nhỏ ảnh làm vật nhỏ tụt xuống dưới một pixel ở tầng thô. Nó nhận chuyển động của nền, và các tầng mịn hơn chỉ tinh chỉnh quanh nghiệm sai đó nên không bao giờ sửa lại được.
●●● Liên kết dữ liệu	Bước quyết định phép đo nào ở khung này ứng với đối tượng nào đã biết. Trong tracking là ghép hộp với track, trong SLAM là ghép đặc trưng với điểm bản đồ. Đây là chỗ hệ thống hồng, chứ không phải bước phát hiện.
●● HOTA	Thước đo tracking tách riêng chất lượng phát hiện (DetA) và chất lượng liên kết (AssA), rồi lấy trung bình nhân $\sqrt{\text{DetA} \cdot \text{AssA}}$. Vì là trung bình nhân, không thành phần nào bù đắp được cho thành phần kia.
●● Vì sao MOTA che được lỗi đổi mã số?	Vì phát hiện sót và phát hiện thừa đếm trên từng khung cho từng vật, còn mỗi lần đổi mã chỉ đếm một lần. Với 1000 vật-khung, giảm số lần đổi mã từ 40 xuống 20 chỉ nâng MOTA từ 0,88 lên 0,90.
●●● Cửa sổ nhân quả	Toàn bộ dữ liệu tới thời điểm hiện tại mà một hệ trực tuyến được phép dùng. Nó loại thẳng mọi kiến trúc hai chiều và mọi phép làm mượt nhìn về phía trước. Thêm nữa, độ trễ xử lý khiến câu trả lời luôn nói về một quá khứ gần.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● mIoU từng khung cao nhưng video nhấp nháy — nghĩa là gì?	Mô hình chạy độc lập trên từng khung và hàm mất mát không phạt việc đổi dự đoán giữa hai khung gần giống nhau. Phân biệt với lỗi phát hiện bằng cách đo độ ổn định giữa các khung liên tiếp, không đo mIoU.
● Bám mất dấu sau khi vật bị che vài giây — nguyên nhân?	Độ bất định của mô hình chuyển động phình ra, nên cổng lọc hoặc quá rộng và ghép bừa, hoặc quá hẹp và mất dấu. Không có giá trị ngưỡng nào thoát khỏi đánh đổi này; chỉ thêm thông tin ngoại hình mới cứu được.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
RAFT và các bản kế thừa	Khối tương quan toàn cặp cộng tính chính lập; số vòng lặp làm núng vận độ trễ	Vẫn là kiến trúc tham chiếu của dòng quang học sâu tính tới 9/2026
ByteTrack, BoT-SORT	Tracking-by-detection: bộ lọc Kalman dự đoán, thuật toán Hungary ghép cặp, và ý “đừng vớt phát hiện điểm thấp”	Được dùng rộng rãi trong hệ đếm và giám sát thực tế
TrackEval	Hiện thực HOTA — tách chất lượng phát hiện khỏi chất lượng liên kết thay vì gộp vào một số	Hạ tầng đánh giá; bền hơn mọi tên mô hình trong danh sách này
SAM 3 (2026) cho video	Phân vùng theo prompt cộng bám danh tính qua khung hình — đúng bài toán liên kết dữ liệu, chỉ khác là đơn vị được liên kết là mask	Ranh giới giữa segmentation và tracking đang mờ dần
co-tracker	Bám điểm dày qua thời gian; nối trực tiếp bài toán tương ứng điểm với bài toán giữ danh tính	Hữu ích khi bạn cần tương ứng dài hạn chứ không chỉ giữa hai khung liên tiếp
Kiến trúc SlowFast và các bản kế thừa	Ý “ngữ nghĩa thay đổi chậm, chuyển động thay đổi nhanh nên không cần cùng tốc độ lấy mẫu”	Vẫn xuất hiện trong nhiều mô hình video hai nhánh; tên mô hình dẫn đầu thì đổi liên tục

MINI PROJECT SLAM · Dòng quang hay mô tả đặc trưng — ngưỡng chuyển động nào làm mỗi bên gãy? · 1–2 ngày

Mục tiêu	đo trực tiếp thứ mà cả chương chỉ nói bằng lời: hai cách giải bài toán tương ứng điểm — bám theo dòng quang và ghép theo mô tả đặc trưng — gãy ở đâu, và ngưỡng gãy đó nằm ở độ lớn chuyển động nào. Đây chính là quyết định liên kết dữ liệu mà frontend SLAM của bạn phải đưa ra ở mỗi khung hình.
Dữ liệu	một chuỗi EuRoC có dải chuyển động rộng (chuỗi “khó” như V2_03 là lựa chọn tốt vì nó có cả đoạn chậm lẫn đoạn lắc mạnh và mờ). Chân trị tương ứng điểm không có sẵn, nên dùng <i>kiểm tra vòng</i> làm thước đo thay thế: ghép từ khung t sang $t + 1$ rồi ghép ngược lại, cặp nào quay về đúng vị trí ban đầu trong ngưỡng một pixel thì tính là ghép đúng.
Các bước	(1) trích một tập đặc trưng góc chung cho cả hai phương pháp trên mỗi khung hình, để hai bên xuất phát từ cùng một tập điểm; (2) nhánh A: bám các điểm đó sang khung sau bằng dòng quang (KLT có kim tự tháp, hoặc RAFT rồi lấy mẫu tại các điểm); nhánh B: tính mô tả đặc trưng và ghép theo khoảng cách gần nhất có kiểm tra tỉ số; (3) với mỗi cặp khung hình, tính độ lớn chuyển động trung vị (từ chân trị tư thế, hoặc từ chính trung vị dòng quang) và tỉ lệ ghép đúng theo kiểm tra vòng; (4) chia các cặp khung hình thành các nhóm theo độ lớn chuyển động và vẽ tỉ lệ ghép đúng theo nhóm cho cả hai nhánh.

- Xong khi** bạn có một đồ thị tỉ lệ ghép đúng theo độ lớn chuyển động, hai đường cho hai phương pháp. Bạn chỉ ra được bằng số ngưỡng mà mỗi đường bắt đầu sụt, tính bằng pixel dịch chuyển trung vị. Và bạn giải thích được cơ chế: dòng quang gây vì khai triển bậc nhất hết hiệu lực và kim tự tháp làm mất vật nhỏ, còn mô tả đặc trưng gây vì mờ chuyển động phá kết cấu cục bộ.
- Nối với** C/15/03 cho cơ chế của cả hai bên; A/01 cho lý do ma trận cấu trúc quyết định điểm nào bám được; E/24 cho cách trình bày kết quả; dùng lại đúng chuỗi và đúng harness của mini project chương 14.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hàm mất mát của mô hình per-frame không có số hạng nào phạt việc dự đoán đổi qua đổi lại giữa hai khung gần như giống nhau; và mIoU tính độc lập trên từng khung nên trung bình của nhiều khung nhấp nháy vẫn cao. Muốn thấy lỗi đó phải đo sự ổn định *giữa* các khung, thứ chưa được chuẩn hoá thành một thước đo phổ biến.
- A2. Nó ra đời để gỡ nghẽn cổ chai thông tin: cả ảnh bị nén vào một vector trước khi biết câu sẽ nói gì. Nút thắt này không xuất hiện ở phân loại vì ở đó đầu ra chỉ có một câu hỏi duy nhất và một vector là đủ; nó chỉ lộ ra khi đầu ra gồm nhiều bước, mỗi bước cần một phần khác nhau của ảnh.
- A3. Vì $MOTA = 1 - (FN + FP + IDSW)/GT$ đếm phát hiện sót và phát hiện thừa *trên từng khung cho từng vật*, nhưng mỗi lần đổi mã số chỉ được đếm một lần. Với nghìn vật–khung, giảm một nửa số lần đổi mã chỉ nâng MOTA vài phần trăm điểm. HOTA sửa bằng trung bình nhân $\sqrt{DetA \cdot AssA}$, nên không thành phần nào bù cho thành phần kia.
- A4. Vì trục thời gian có hướng: đảo ngược nó thì “mở” thành “đóng”, trong khi lật trục x vẫn cho một ảnh hợp lệ. Nó lại còn có đơn vị khác hẳn hai trục kia. Vậy không có lý do gì để cùng một kernel $3 \times 3 \times 3$ là kích thước hợp lý ở cả ba trục. Hai bất đối xứng đó là nguyên nhân sâu xa của mọi thiết kế tách trục.
- A5. Bằng độ phân giải hoặc bằng batch, theo quan hệ $R \propto 1/\sqrt{TB}$. Ba cách không tương đương: giảm batch làm hỏng thống kê của chuẩn hoá theo batch, giảm độ phân giải làm mất vật nhỏ vĩnh viễn, còn giảm độ dài clip thì làm mất chính thông tin thời gian mà bạn đang trả tiền để có.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn ngồi trước một cái bàn nhỏ, và cứ mỗi một phần ba mươi giây lại có người đặt thêm lên bàn một tấm ảnh chụp cùng một hành lang. Nhìn một tấm ảnh, bạn nói được “có một người” — nhưng để nói “người đó đang bước tới” hoặc “vẫn là người lúc này chứ không phải người khác”, bạn buộc phải đặt vài tấm cạnh nhau và so.

Mẹo thì đơn giản: so từng chỗ trên tấm này với chỗ tương ứng trên tấm kia. Hoặc so từng điểm nhỏ, để biết mọi thứ dịch đi đâu; hoặc so cả một vùng, để nói được “cái hộp này” ở tấm trước với “cái hộp kia” ở tấm sau. Việc nói đó, chứ không phải việc nhìn thấy, mới là phần khó.

Cái giá gồm ba khoản, và cả ba đều đắt. Thứ nhất, mặt bàn có hạn: muốn để bốn mươi tấm thay vì mười tấm thì phải in ảnh nhỏ lại, mà ảnh nhỏ thì mất những vật bé. Thứ hai, các tấm ảnh của tương lai chưa được chụp, nên bạn chỉ được dùng những gì đã có tới lúc này — mọi cách làm cần “xem hết rồi mới nói” đều bị loại ngay. Thứ ba, đến khi bạn nói xong thì người trong ảnh đã bước thêm mấy bước, nên câu trả lời của bạn luôn nói về một quá khứ gần.

Chỗ không rút gọn được: ngưỡng chuyển động mà một cách nối bắt đầu sai phụ thuộc vào kết cấu bề mặt, độ mờ và cách phơi sáng của đúng cái camera bạn dùng, nên không có con số chung để nhớ — chỗ này bắt buộc phải tự đo.

Một câu: Thêm thời gian vào ảnh không tặng bạn một chiều dữ liệu, nó gửi cho bạn ba hoá đơn: bộ nhớ nhân lên, câu trả lời luôn muộn, và phần khó nhất là nối đúng vật ở tấm này với vật ở tấm kia.

Tổng kết chương

Ba vấn đề mà chiều thời gian sinh ra hoá ra có ba câu trả lời rất khác nhau về bản chất. Đầu ra độ dài thay đổi được giải bằng một *mẫu thiết kế*: giữ nguyên nguồn thông tin và hoãn việc nén lại cho tới khi có truy vấn — và mẫu thiết kế đó, dưới cái tên attention, sau này chiếm lĩnh cả lĩnh vực. Chi phí bộ nhớ không có lời giải, chỉ có kế toán: tích của độ dài clip, độ phân giải bình phương và batch size gần như là hằng số, nên mọi kiến trúc là một cách chọn hy sinh. Ràng buộc nhân quả cũng không có lời giải, nó chỉ đơn giản loại bỏ phương án — và loại bỏ trước khi ta kịp bàn tới độ chính xác.

Bài học chung nhất thì nằm ở phần liên kết. Trong optical flow cũng như trong tracking, phần khó không phải nhìn thấy mà là ghép cặp, và trong cả hai lĩnh vực đều tồn tại một thước đo phổ biến che giấu đúng phần khó đó. Đây là biến thể của bài học ở chương 14: chi tiết không được viết trong abstract thường quyết định kết quả. Ở đây nó xuất hiện dưới dạng thước đo, không phải dưới dạng cách gán mẫu. Thói quen cần mang theo là như nhau: trước khi tin một con số, hãy hỏi nó cộng những loại lỗi nào lại với nhau và với trọng số bao nhiêu.

Còn bỏ ngỏ hai hướng. Thứ nhất, toàn chương này làm việc trên mặt phẳng ảnh; khi bài toán tương ứng điểm được nâng lên không gian ba chiều với ràng buộc epipolar và độ sâu, ta bước sang C/16-thi-giac-3d — và đó là chỗ mà nền hình học của bạn phát huy trực tiếp nhất. Thứ hai, mọi mô hình ở đây đều cần nhân thời gian đắt đỏ; C/18-pretraining-va-foundation bàn về việc lấy giám sát từ chính cấu trúc thời gian của video, thứ có sẵn miễn phí.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba vấn đề riêng của chiều thời gian — đầu ra độ dài thay đổi, chi phí bộ nhớ, ràng buộc nhân quả — có xung đột với nhau không? Chỉ ra một thiết kế mà chữa vấn đề này thì làm nặng thêm vấn đề kia.
2. Mục 1 sinh ra attention để tránh nghẽn “nén trước, dùng sau”, còn mục 2 cho thấy chính attention nổ chi phí trên clip dài. Hai mục vì thế đẩy về hai hướng ngược nhau; bạn hoà giải thế nào cho một mô hình chạy trực tuyến?
3. Tương ứng điểm và liên kết dữ liệu là hai bài toán mà một hệ SLAM đã giải ở mỗi khung hình. Với mỗi bài, phần nào của từ vựng SLAM dịch thẳng sang được và phần nào không, và chỗ không dịch được bắt nguồn từ giả định nào?
4. Nhấp nháy không xuất hiện trong mIoU, đổi danh tính gần như không xuất hiện trong MOTA, và độ trễ không xuất hiện trong “độ chính xác trên khung t ”. Ba lỗ hổng đó có chung một nguyên nhân về cách xây thước đo không, và nó là gì?
5. Cửa sổ trượt, trạng thái hồi quy và lấy mẫu thưa theo thời gian là ba cách trả giá cho việc kéo dài clip. Chúng hỏng theo ba kiểu khác nhau nào, và loại chuyển động nào phơi bày từng kiểu?
6. Baseline chỉ nhìn một khung hình rồi gộp lại gần bằng mô hình thời gian đắt hơn nhiều. Trước khi kết luận rằng thời gian không quan trọng, bạn kiểm ba khả năng nào ở dữ liệu và ở thước đo?

Nguồn

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] K. Xu, J. Ba, R. Kiros, et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” in *ICML*, 2015.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” in *ICLR*, 2015.
- [4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [5] R. Mur-Artal and J. D. Tardós, “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras,” *IEEE T-RO*, vol. 33, no. 5, pp. 1255–1262, 2017.
- [6] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [7] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [8] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [9] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [10] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [11] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [12] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [13] A. Dosovitskiy, P. Fischer, E. Ilg, et al., “FlowNet: Learning Optical Flow with Convolutional Networks,” in *ICCV*, 2015.
- [14] D. Sun, X. Yang, M.-Y. Liu, et al., “PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume,” in *CVPR*, 2018.
- [15] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” in *ECCV*, 2020.
- [16] A. Bewley, Z. Ge, L. Ott, et al., “Simple Online and Realtime Tracking,” in *ICIP*, 2016.
- [17] Y. Zhang, P. Sun, Y. Jiang, et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *ECCV*, 2022.
- [18] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [19] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP J. Image and Video Processing*, 2008.